

Chapter 01

Introduction to Flet and Python Setup

4 lessons • 25 hr

Chapter 01

4 lessons • 25 hr

LESSON 01	Installing Python and Flet	14 min	4
	Summary	6 min	6
	Lesson transcript	6 min	7
	Further study materials	8 min	10
	Exercises		14
LESSON 02	What Is Flet and Why Use It	54 min	19
	Summary	6 min	21
	Lesson transcript	6 min	22
	Further study materials	47 min	26
	Exercises		54
LESSON 03	Configuring Your Code Editor	1 hr 25 min	64
	Summary	7 min	66
	Lesson transcript	7 min	67
	Further study materials	1 hr 18 min	70
	Exercises		118

Chapter 01

4 lessons • 25 hr

CONCLUSION	Practical project of the chapter	135
	Context and Provocation	137
	Development Guide	140
	Self-Assessment Rubric	140
	Model Response & Assessment Reference	141
	Notes	147

Chapter 01 • Introduction to Flet and Python Setup

Lesson 01 • Installing Python and Flet

13 pages • 14 min

Lesson 01 • Installing Python and Flet

14 min • 13 pages

© What you will learn

Guides installation of Python 3.8+ and the Flet package via pip. Ensures every student has a verified, functional toolchain before writing code.

Summary	6 min	6
Transcript	6 min	7
Further study materials		
Text • Setting Up a Python Virtual Environment and Resolving Common Issues	8 min	10
Exercises		14
Answer key		18

Installing Python and Flet



Summary

Setting Up Your Flet Development Environment

To start coding with Flet, you need a solid setup. This lesson covers four steps: install compatible Python, create a virtual environment, install Flet using pip, and confirm everything works. By completing these steps, you will have a clean development environment ready for building apps. Follow each step carefully to avoid common issues.

Check and Install Python

Flet requires Python 3.8 or higher, but 3.11 or 3.12 are best. Open your terminal and type 'python --version' on Windows or 'python3 --version' on Mac/Linux. If the version is 3.8 or higher, you are good. If not, download the latest stable release from python.org. This step ensures your system has the right Python to run Flet smoothly.

Create a Virtual Environment

A virtual environment keeps your project's packages separate from other projects. Navigate to your project folder in the terminal. Run 'python -m venv venv' to create it. Then activate it: on Mac/Linux use 'source venv/bin/activate', on Windows use 'venv\Scripts\activate'. Your prompt will show '(venv)'. Now any package you install goes into this isolated space, avoiding version conflicts.

Install Flet with pip

With the virtual environment active, install Flet by typing 'pip install flet' and pressing Enter. Pip will download Flet and its dependencies, including flet-core and httpx. After a few seconds, you will see a success message. No extra steps needed – everything is bundled. Now you have the Flet framework ready to build Python user interfaces.

Verify Installation and Next Steps

Run 'pip show flet' to see the version and location. Or run 'python -c "import flet; print(flet.version.version)"' to confirm it works. If you see a version number, your setup is ready. Common issues: pip not found (try pip3), not activating the virtual environment, or using sudo in venv. Use 'deactivate' to exit the environment. Follow the checklist: Python version, venv active, Flet installed, verified. Now you are ready to build your first app.

NOTES



Installing Python and Flet

Transcript

Introduction

You need a solid foundation before writing any Flet code. This lesson covers it: you'll install Python, set up a virtual environment, install Flet, and confirm everything works correctly. We'll go step by step, so by the end you'll have a clean development environment ready for the next lesson.

Here's the plan for this lesson: four steps. Start by installing a compatible Python version. Then create a virtual environment to isolate your project. Install Flet using pip. Finally, run a quick check to confirm everything works. Complete these four steps, and you'll have a rock-solid setup for every lesson that follows in this course.

Checking Python Version

Flet requires Python 3.8 or higher, but go with 3.11 or 3.12 for the best compatibility and performance. They're stable, fully supported, and run Flet's latest features without a hitch. If you're on an older Python, upgrade now. Next, we'll check which version you have installed.

Open your terminal. On Windows, type `python --version`. On macOS and Linux, type `python3 --version`. Press Enter and you'll see the version number appear. If it shows 3.8 or higher, you're all set. If it's lower or you get a "command not found" error, go to python.org and download the latest stable release for your operating system. Make sure you type the command exactly as shown—lowercase, with two dashes before 'version'. This quick verification prepares you for the next steps.

Setting Up a Virtual Environment

A virtual environment isolates your project's dependencies from everything else on your machine. Without one, installing packages globally leads to version conflicts between projects and makes reproduction difficult. With one, each project gets its own clean space — no conflicts, fully reproducible. It's a small habit that saves you from huge headaches.

To create a virtual environment, you only need three commands. Start by navigating to your project folder in the terminal. Then run `python -m venv venv` to create the environment. Finally, activate it. On Mac and Linux, use `source venv/bin/activate`. On Windows, it's `venv\Scripts\activate`. After activation, your terminal prompt will show the environment name.

Once activated, your prompt shows the environment name in parentheses — that `(venv)` prefix right before your usual path. That's your confirmation it's running. Any package you install now goes into this isolated environment, not your global Python. With that set, let's install Flet using pip.

NOTES

Installing Python and Flet

Transcript

Installing Flet

With your virtual environment active, installing Flet is a single command. Type `pip install flet` and press Enter. pip will download Flet and all its dependencies – you'll see progress bars as it fetches each package. The whole process takes just a few seconds, depending on your connection speed. Once the success message appears, the installation is complete. That final "Successfully installed flet" line confirms everything worked.

Run `pip install flet`, and you get more than just the core framework. Pip resolves the dependency tree, installing flet-core with low-level controls and rendering engine, plus httpx and other supporting libraries. No manual installs needed. The Flet team bundled everything so you can start building your Python UI right away.

Verifying the Installation

After installation, confirm it works with two simple checks. Run `pip show flet` — you'll see the package version and where it's stored on disk. Alternatively, run `python -c "import flet; print(flet.version.version)"` to verify Python can load it. Both commands should print a version number. If you get a clean output with no errors, Flet is ready. You can start building right away.

Here's a quick sanity check. Open a Python file, write `import flet as ft`, then print the version number. Run it. If you see a version like 0.24.1 printed with no errors, your environment is completely ready. Two lines of code confirm Python, pip, and Flet are all set up correctly. And that exact import pattern? It's what you'll use in every Flet app you build in this course. So go ahead, give it a try.

Troubleshooting Common Issues

The most common hiccup: 'pip not found'. If that happens, try `pip3` instead, or double-check that Python is in your system PATH. Another frequent mistake: installing with the wrong Python version. That usually means you didn't activate the virtual environment before running pip. And finally, permission errors. Inside a virtual environment, never use `sudo` with pip. The venv already handles permissions; `sudo` only creates unnecessary conflicts.

When you finish a session, just type `deactivate` in your terminal to leave the virtual environment. The prompt prefix vanishes, and you're back in your global Python. Before your next session on this project, `cd` into its folder and reactivate the environment first — then run your code.

Best Practices: Project Structure

NOTES

Installing Python and Flet

Transcript

A clean project structure from the start saves headaches. Your root folder holds the project. Inside it, the `venv` folder contains the virtual environment — never commit that to git. Your main Python file, usually named `main.py`, sits at the root level. Add a `.gitignore` file and exclude the `venv` folder from version control.

Summary Checklist

Before moving on, run through this checklist. First, confirm Python 3.8 or higher with `python --version`. Second, create your virtual environment with `python -m venv venv`, activate it — look for the `(venv)` prompt. Third, install Flet with `pip install flet`, verify with `pip show flet`. If all four check out, your development environment is solid and you're ready to build.

Conclusion

You've ticked all four boxes: Python installed, virtual environment created and active, Flet installed and verified. Your environment is fully configured. That setup is the foundation for everything ahead. Next, you'll write your first Flet app and watch it run live on your screen.

NOTES

Further study materials Lesson 1

Content type: Text

Setting Up a Python Virtual Environment and Resolving Common Issues

Available at
<https://medium.com/@santiagovahos/qu%C3%A9-es-un-entorno-virtual-de-python-y-por-qu%C3%A9-es-importante-9aff606f4414>

Full
Content



Summary

What is a Python Virtual Environment?

Imagine building a house where you plan and use specific materials. Similarly, a Python virtual environment is a separate room in your computer for each project. In this room, you install only the packages and libraries needed for that project. This keeps your projects organized and avoids conflicts. The analogy of building a house helps understand the importance of planning and using the right materials. Virtual environments isolate dependencies, ensuring changes in one project do not affect others.

Why Use a Virtual Environment?

Without virtual environments, installing different versions of the same library for different projects causes conflicts. For example, one project needs an older version and another a newer version of the same library. This can break your projects. Virtual environments allow each project to have its own set of packages, so you can work on multiple projects without interference. They also help share projects easily because the exact dependencies are recorded.

Creating a Virtual Environment

To create a virtual environment, first create a folder for your project. Then navigate to it in Git Bash. Run `'python -m venv environment_name'` to create it. Activate it with `'source environment_name/Scripts/activate.bat'`. Finally, open Visual Studio Code by typing `'code .'`. This sets up an isolated environment for your project. Remember to replace `'environment_name'` with your desired name. These steps ensure you have a clean workspace to install packages.

Activate in VS Code and Common Error

After opening VS Code from the terminal, press `Ctrl+Ñ` to open the built-in terminal. You may see an error: `'execution of scripts is disabled on this system'`. This prevents the virtual environment from activating. To fix it, you need to change the execution policy in PowerShell. Open PowerShell as administrator and check the current policy with `'Get-ExecutionPolicy -list'`. The policy might be

NOTES

Further study materials Lesson 1



Summary

'Undefined' for LocalMachine.

Fixing Script Execution Policy

To fix the execution policy, run 'Set-ExecutionPolicy RemoteSigned -Force' in an administrator PowerShell. This changes LocalMachine to RemoteSigned. Then open 'gpedit.msc' (Windows+R) and navigate to Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell. Enable 'Turn on Script Execution' and select 'Allow only signed scripts'. Apply and OK. Close the old terminal and open a new one in VS Code. The virtual environment name should appear in the prompt.

NOTES



Further study materials Lesson 1

Transcript

What is a Python Virtual Environment and Why is it Important?

Imagine you want to build a house. Before you start building, you need to plan everything and buy the materials. For the house to be well built, you need to have a plan and use the right materials.

The same thing happens when you work on a Python project. Before you start programming, you need to have a plan and know what materials (Python packages and libraries) you are going to use. This is where virtual environments come into play.

A virtual environment is like a room you create inside your computer to work on a Python project. In this room, you can install the specific packages and libraries you need for your project, without affecting other rooms (projects) you may have on your computer.

Imagine you have two Python projects that use the same library, but one of them needs an older version and the other needs a newer version. If you don't use a virtual environment, you will have trouble installing both versions on your computer, and the projects might not work correctly.

That is why it is important to use a virtual environment. It allows you to work on a Python project with the specific packages and libraries you need, without affecting other projects or your operating system. Additionally, it helps you keep your project organized and allows you to share it with other people without worrying about differences in library versions.

Now that you know its importance, it's time to learn how to create one. Below, I'll show you how to create and activate a virtual environment in Python from Git Bash for use in Visual Studio Code:

```
# 1. First, create the directory that will contain the
virtual environment
mkdir directory_name

# 2. Navigate to the directory and run "python -m venv"
to create the virtual environment
cd folder_name
python -m venv environment_name

# 3. Activate the virtual environment using the "source"
command
source environment_name\Scripts\activate.bat

# 4. Finally, open Visual Studio Code with the command
"code ."
code .
```

You can open the terminal from VS Code with the key combination

NOTES

Further study materials Lesson 1

Transcript

"Ctrl + Ñ" to check if you see the following error: "execution of scripts is disabled on this system". If so, the first step to fix it is to open the PowerShell terminal as administrator, then run the command "Get-ExecutionPolicy". This command will show you the current execution policy on your system as shown below:

```
PS C:\Users\username> Get-ExecutionPolicy -list
Scope ExecutionPolicy
-----
MachinePolicy Undefined
UserPolicy Undefined
Process Undefined
CurrentUser Undefined
LocalMachine Undefined
```

You need to change the state of LocalMachine with the following command:

```
PS C:\Users\username> Set-ExecutionPolicy RemoteSigned
-Force
```

To verify the state of LocalMachine, run again:

```
PS C:\Users\username> Get-ExecutionPolicy -list
Scope ExecutionPolicy
-----
MachinePolicy Undefined
UserPolicy Undefined
Process Undefined
CurrentUser Undefined
LocalMachine RemoteSigned
```

The second step is to open gpedit.msc from the "Run" menu by pressing "Windows + R". If you don't have gpedit.msc installed, **here you will find a tutorial that will teach you how to install it.** Once in the gpedit.msc window, look for the "Windows PowerShell" option located under "Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell". Here you will select "Turn on Script Execution" and click "Enabled". Then select "Allow only signed scripts" in the execution policy and finally click "Apply" and "OK". With this, your virtual environment will be correctly activated.

To verify what was done, you must close the previous terminal and open a new one by pressing "Ctrl + Ñ". If everything has worked correctly, you should not see the error that appeared before, and instead, the name of the virtual environment should appear at the beginning of the terminal prompt like this:

```
(environment_name) PS C:\Users\username\directory_name
```

NOTES

Exercises • Lesson 1

EXERCISE 1 • True or false

CONTEXT

This lesson covers the initial steps to set up a development environment for Flet.

QUESTION • Indicate whether the information is true or false

A virtual environment isolates your project's dependencies from other projects, preventing version conflicts.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 2 • Multiple choice

CONTEXT

After creating a virtual environment to isolate project dependencies, you need to activate it before installing packages. The activation command varies by operating system.

QUESTION • Select the correct option

Which command activates the virtual environment on Windows?

- ☐ A `source venv/bin/activate`
- ☐ B `activate`
- ☐ C `venv/bin/activate`
- ☐ D `venv\Scripts\activate`

NOTES

Exercises • Lesson 1

EXERCISE 3 • True or false

CONTEXT

When working on multiple Python projects, each project may require different versions of the same library. Managing these dependencies properly is essential to avoid conflicts.

QUESTION • Indicate whether the information is true or false

Using a Python virtual environment allows you to install specific packages for one project without affecting other projects on the same computer.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 1

EXERCISE 4 • Multiple choice

CONTEXT

After creating a virtual environment in Python using Git Bash, you need to activate it before using it. The activation command varies depending on your terminal.

QUESTION • Select the correct option

Which command correctly activates a Python virtual environment created in Git Bash?

- ☐ A `source environment_name\Scripts\activate`
- ☐ B `source environment_name\Scripts\activate.bat`
- ☐ C `activate.bat`
- ☐ D `python -m venv environment_name`

NOTES

Exercise Answers

LESSON 1

EX.01 • True or false

True.

Virtual environments create isolated spaces for each project, so packages installed in one do not affect others, avoiding version conflicts.

☒ I got it right ☐ I got it wrong

EX.02 • Multiple choice

Option D — “venv\Scripts\activate”

The lesson explicitly states that on Windows, the activation command is 'venv\Scripts\activate'. This sets up the isolated environment and changes the prompt to show '(venv)'.

☒ I got it right ☐ I got it wrong

EX.03 • True or false

True.

A virtual environment creates an isolated environment where you can install dependencies specific to a project, preventing conflicts with other projects or system-wide packages.

☒ I got it right ☐ I got it wrong

EX.04 • Multiple choice

Option B — “source environment_name\Scripts\activate.bat”

This command runs the activation script for the virtual environment in Git Bash, as shown in the lesson.

☒ I got it right ☐ I got it wrong

Chapter 01 • Introduction to Flet and Python Setup

Lesson 02 • What Is Flet and Why Use It

43 pages • 54 min

Lesson 02 • What Is Flet and Why Use It

54 min • 43 pages

© What you will learn

Covers Flet's role as a Flutter-based Python UI framework and its advantages over alternatives. Sets the conceptual foundation for all subsequent development work.

Summary	6 min	21
Transcript	6 min	22
Further study materials		
Text • How to Use Flet to Build Cross-Platform Apps with Python and Flutter	24 min	26
Text • Build a Flutter-Style Counter App with Python and the Flet Framework	11 min	39
Text • Comparing Python GUI Frameworks for Efficient Automation	11 min	46
Exercises		54
Answer key		62

What Is Flet and Why Use It

Summary

What Is Flet and the Problem It Solves

Flet is a framework that lets you build apps for desktop, mobile, and web from a single Python codebase. It handles state management, layout, and packaging so you only write Python. Python is great for logic but building cross-platform apps with polished UIs has been painful. Existing tools force ugly trade-offs like clunky interfaces, steep learning curves, or platform limits. Flet closes this gap, giving Python developers the cross-platform UI they deserve.

Flet vs. Tkinter and PyQt

Tkinter comes with Python but looks outdated and lacks mobile/web support. PyQt offers polished UIs but has a steep learning curve and no mobile deployment. Flet provides modern UIs, built-in mobile and web support, and a gentle learning curve using plain Python. This combination of modern, cross-platform, and simple is rare. Flet makes it easy to build apps for all platforms without extra frameworks.

Architecture and Client-Server Model

Flet is not a wrapper around HTML or native widgets. It sits on top of Flutter, Google's high-performance UI engine. You write Python, and Flutter draws the pixels. The architecture has layers: GPU renderer, Flutter widgets, and Flet wrapping them as Python objects. Flet uses a client-server model: Python script acts as server with state and events, Flutter client renders UI and captures input. This separation makes porting easy.

One Code, Six Targets and Use Cases

Flet apps run on Windows, macOS, Linux, Android, iOS, and the web from one Python file. During development, run as desktop; ship for mobile; deploy to web without code changes. Ideal for internal tools, cross-platform apps, rapid prototypes, and data-driven apps. Not for tiny utilities due to larger bundle size and need for Android Studio/Xcode. Community is smaller than PyQt but growing.

Course Coverage and Key Takeaways

The course starts with Hello World, then multi-screen apps with navigation and state, connecting to APIs, adding SQLite databases, and packaging for all platforms. Key takeaways: Flet uses Flutter's GPU-accelerated UI in Python. Client-server model separates logic from rendering. One codebase deploys across six platforms. Everything you know about Python works directly. Flet gives your code a high-quality visual front end.

NOTES

What Is Flet and Why Use It

Transcript

What Is Flet?

You already have your environment set up. So what is Flet? Why should you care? It's a framework that lets you build apps for desktop, mobile, and web — all from a single Python codebase. That's the promise.

It handles the hard parts — state management, layout, packaging — so you just write Python. But to really get why it matters, we need to start with the problem it solves.

The Problem It Solves

Every Python developer has felt this frustration. The language is brilliant for logic — readable, fast, expressive. You master complex algorithms and data pipelines with ease.

But building a real app with a polished interface that runs on Windows, Mac, mobile, and the web? That's always been where the pain begins. The existing tools force ugly trade-offs: clunky UIs, steep learning curves, or platform lock-in.

That gap has held Python back for too long. Flet was built to close it — finally giving you the cross-platform UI you deserve.

Flet vs. the Competition

Let's put Flet next to its main competitors. Tkinter is the classic choice — it comes with Python, works fine, but the UI looks outdated and there's no support for mobile or web.

PyQt gives you a polished, professional interface, but you pay for it with a steep learning curve, and mobile deployment is still absent.

Flet offers a modern UI, built-in mobile and web support, and a gentle learning curve because you're writing plain Python — no extra frameworks to learn. That combination — modern, cross-platform, and simple — is rare.

How Flet Works Under the Hood

The key insight is this. Flet is not a Python wrapper around HTML or a native OS widget toolkit. Instead, it sits on top of Flutter, Google's high-performance UI engine.

That means your application renders with the exact same engine used by millions of production mobile apps. You write Python, and Flutter draws the pixels. It's that straightforward.

The Layered Architecture

Picture it as concentric layers. At the very core is Skia or Impeller — Flutter's GPU renderer that draws every pixel. Flutter sits on top, providing widgets and layout.

Flet then wraps Flutter and surfaces everything as Python objects.

NOTES

What Is Flet and Why Use It

Transcript

You stay at the outermost layer, writing Python classes and functions. Everything beneath handles the rendering automatically, no manual GPU work required.

Client-Server Model for Portability

Flet uses a client-server model even when running locally: your Python script acts as the server, holding the app state and responding to events. The Flutter client is the visual front end that renders controls and captures user input.

When a user clicks a button, that event travels back to your Python function. This separation is what makes Flet flexible enough to run as a desktop app, a web app, or a mobile app with almost no code changes.

In a traditional GUI, your logic and interface are tangled. Porting to a new platform means untangling everything. Flet keeps the Python logic separate from the rendering client.

Point the same script at a desktop window, a browser tab, or a mobile screen. Portability becomes a default, not an afterthought.

Where Can You Run a Flet App?

The architecture gives you portability by default, so where can you actually run a Flet app? Windows, macOS, Linux, Android, iOS, and the browser.

That's a far wider reach than most Python developers expect — six deployment targets in one shot.

So where can you run a Flet app? Across six platforms. It's not just one or two — Flet covers all three major deployment categories.

On desktop, you package your app as a standalone executable for Windows, macOS, or Linux. On mobile, Flet compiles your app to a native Android APK or iOS IPA.

On the web, your app runs as a Flutter web application, served by any Python web server. One codebase, six targets — exactly what we'll look at next.

Practical Payoff: One Code, Six Targets

This is the practical payoff. You write one Python file and that same code runs on six targets: desktop, mobile, and web.

During development, run it as a desktop app. When you're ready to ship, package it for mobile. For users who don't want to install anything, deploy it to the web.

The code never changes. Only the build command does.

Ideal Use Cases

NOTES

What Is Flet and Why Use It

Transcript

Flet shines in four situations. Internal tools — build real UIs without a web stack. Cross-platform apps — one codebase for desktops and mobile.

Rapid prototypes — Python speed with production-quality UI. And data-driven apps — pair your pandas or SQLite with a front end, no language switch needed.

When Not to Use Flet

Flet isn't for every situation. Its bundle is bigger than Tkinter's, important if users have slow connections. Mobile builds need Android Studio or Xcode, adding setup time.

The community is smaller than PyQt's, though growing fast. For a tiny personal utility, Tkinter might be simpler. But for anything that needs polish and cross-platform reach, Flet wins.

Architecture Recap

That's the promise — here's the architecture in three steps. You write app logic in Python: controls, state, event handlers.

The Flet runtime translates those Python objects into Flutter widget instructions. Flutter then draws the final UI with GPU acceleration.

Each layer works independently, and you only touch the top one. And through it all, you're still in Python — more on that next.

You Stay in Python

Let's be direct about this. Everything you already know about Python applies directly. Functions, classes, loops, libraries — all of it works as expected.

Flet does not demand a new paradigm or language. It simply gives your Python code a high-quality visual front end, handling the rest.

What This Course Covers

Here's where this course takes you. You'll start with a classic Hello World — the first Flet app you run.

Then you move into multi-screen apps with proper navigation and state. After that, connect to live APIs to fetch and display data.

Next, add a local database layer with SQLite to persist everything. Finally, package your app so it runs on every platform Flet supports. Each step builds directly on the last.

Key Takeaways

Three key points. Flet uses Flutter's rendering engine for modern, GPU-accelerated UI — all in Python. The client-server model separates logic from rendering, with real-time communication.

NOTES

What Is Flet and Why Use It

Transcript

This allows one codebase to deploy across Windows, macOS, Linux, Android, iOS, and the web.

NOTES

Further study materials Lesson 2

Content type: Text

How to Use Flet to Build Cross-Platform Apps with Python and Flutter

Available at
<https://medium.com/django-unleashed/how-to-build-cross-platform-desktop-apps-in-python-with-flet-cf587cae8914>

Full
Content



Summary

Introduction to Flet and Cross-Platform Apps

The demand for cross-platform desktop apps is high. Python is popular for its simplicity. Flet is a new framework that combines Python with Flutter's UI. It lets developers create apps for Windows, macOS, and Linux easily. This article guides you through building desktop apps with Flet, from setup to advanced features.

Understanding Flet's Core Principles and Comparisons

Flet uses pre-built widgets and a declarative syntax. It handles UI abstraction, state management, and event handling. Compared to Tkinter and PyQt, Flet offers a modern look and easier development. Unlike Electron, Flet is lighter and uses only Python, giving a native-like experience across platforms.

Setting Up Environment and Building Your First App

You need Python 3.7 or later. Install Flet with 'pip install flet'. Organize your project with folders like main.py, components, assets. A simple Hello World app creates a text widget and a button. Clicking the button updates the text. The code imports flet, defines a main function, creates UI elements, and uses an event handler to update the page.

Advanced Concepts: State Management, Responsive Design, APIs, Custom Components

State management uses a counter variable that updates text on button click. Responsive design changes layout based on window width using page.on_resize. API integration fetches data with requests library and displays it. Custom components like CustomCard can be created by extending Container, and themes are set with page.theme for consistent look.

Debugging, Optimization, and Packaging Your App

Common pitfalls include forgetting page.update() and misbinding events. Use print statements for debugging. Optimize with lazy loading of tabs, async operations for long tasks, and batching state updates. Package your app using PyInstaller with 'pyinstaller --onefile main.py' to create a standalone executable for Windows,

NOTES

Further study materials Lesson 2



Summary

macOS, or Linux.

NOTES



Further study materials Lesson 2

Transcript

How to Build Cross-Platform Desktop Apps in Python with Flet

Flet emerges as an innovative framework that leverages the robust UI capabilities of Flutter while maintaining Python's ease of use.

[Image: How to Build Cross-Platform Desktop Apps in Python with Flet]

In today's rapidly evolving software landscape, the demand for cross-platform desktop applications is higher than ever.

Developers and businesses are looking for tools that allow them to build feature-rich, responsive applications that run seamlessly on Windows, macOS, and Linux.

Python has long been favored for its simplicity, readability, and extensive library support, making it an excellent choice for developing such applications.

This blend offers a unique approach to building desktop applications: developers can quickly prototype and iterate on their designs without delving into the complexities of traditional UI development.

By combining Python's straightforward syntax with Flutter's dynamic interface components, Flet empowers developers to create visually appealing and highly interactive applications with minimal overhead.

This article will guide you through the process of building desktop applications using Flet.

Check out a more detailed tutorial in this YouTube video, "Build Applications in Python with Flet":

Understanding Flet

Flet is a versatile framework designed for building interactive multi-user applications across web, desktop, and mobile platforms – all using Python.

It enables developers to harness the power of Flutter's rich UI components while writing code in a language known for its simplicity and rapid development capabilities.

This combination allows for the creation of responsive, scalable, and feature-rich applications without the steep learning curve associated with some traditional UI frameworks.

Core Principles

NOTES

Further study materials Lesson 2

Transcript

Flet abstracts the complexities of UI development by offering pre-built widgets and a declarative syntax that simplifies the construction of application interfaces.

Its key principles include:

- **UI Abstraction:** Developers can work with high-level components rather than dealing directly with lower-level UI code.
- **State Management:** Flet efficiently manages the state of your application, ensuring that UI elements are updated dynamically in response to user interactions.
- **Event Handling:** Built-in event handling mechanisms allow for responsive and interactive interfaces, making it easy to capture and react to user actions like clicks, text inputs, or other events.

Comparisons

When compared to other popular GUI frameworks, Flet stands out for several reasons:

Tkinter & PyQt: While Tkinter offers simplicity and is included with Python, and PyQt provides a robust set of tools for creating professional-grade applications, both can be limiting in terms of modern UI design and cross-platform consistency.

Flet, leveraging Flutter's modern UI paradigms, provides a more contemporary look and a smoother development experience without needing to juggle multiple programming languages.

Electron: Electron enables developers to build desktop applications using web technologies like HTML, CSS, and JavaScript.

However, it often comes with performance overhead and larger application sizes. In contrast, Flet delivers a native-like experience using Python, making it a lighter and more efficient alternative while still offering cross-platform compatibility.

Setting Up the Development Environment

Before diving into Flet development, ensure that your system meets the following prerequisites:

Python Installation: Make sure you have Python 3.7 or later installed on your machine. Python can be downloaded from the official Python website.

Installing Flet

Installing Flet is straightforward using Python's package manager, pip. Follow these steps to get started:

Install Flet using pip:

NOTES

Further study materials Lesson 2

Transcript

```
pip install flet
```

Project Structure

A well-organized project structure is crucial for maintaining and scaling your application. Consider the following layout as a starting point:

```
my_flet_app/
├── main.py # Entry point of the application
├── components/ # Folder for reusable UI components and
    widgets
│   ├── __init__.py
│   └── custom_button.py
├── assets/ # Folder for images, icons, and other static
    files
├── requirements.txt # List of dependencies (including
    Flet)
└── README.md # Project overview and setup instructions
```

Building Your First Desktop App with Flet

To get started, let's create a simple "Hello World" desktop application.

This initial example will introduce you to the basic structure of a Flet app, where you define a main function that handles the page setup and adds UI elements.

UI Components

Flet provides a collection of essential widgets that form the building blocks of your application. Some of these include:

- **Text:** For displaying static text.
- **Buttons (e.g., ElevatedButton):** For triggering actions.
- **Text Fields:** For user input.
- **Containers and Layout Widgets:** For organizing your UI elements in a structured manner.

Event Handling

One of the powerful features of Flet is its straightforward approach to event handling.

You can easily define functions that respond to user actions like button clicks or text input changes.

When an event occurs, you update the relevant UI component, and Flet re-renders the updated state on the screen.

Code Walkthrough

Below is an annotated code snippet that demonstrates a basic Flet application.

NOTES

Further study materials Lesson 2

Transcript

This example displays a greeting message and includes a button that, when clicked, updates the greeting:

```
import flet as ft

def main(page: ft.Page):
    # Set the title of the application window
    page.title = "Hello World App"
    # Create a text widget to display a greeting message
    greeting = ft.Text("Hello, World!")

    # Define an event handler for the button click event
    def button_clicked(e):
        # Update the greeting text upon button click
        greeting.value = "Button Clicked! Welcome to Flet."
        # Refresh the page to reflect the updated UI
        page.update()
    # Create an elevated button widget with an associated
    click event handler
    click_button = ft.ElevatedButton(text="Click Me",
    on_click=button_clicked)

    # Add the greeting text and button to the page layout
    page.add(greeting, click_button)

    # Start the Flet application by specifying the main
    function as the target
    ft.app(target=main)
```

Code Explanation:

- **Initialization:** The application starts by importing Flet and defining the `main` function that takes a `page` object. This `page` represents the app window.
- **UI Components:** A `Text` widget is created to display "Hello, World!" and an `ElevatedButton` is set up with the label "Click Me."
- **Event Handling:** The `button_clicked` function changes the text of the `greeting` widget when the button is clicked, and `page.update()` is called to refresh the UI.
- **Rendering:** Finally, both the text and button are added to the page layout using `page.add()`, and the app is launched with `ft.app(target=main)`.

You can run the app with:

```
python main.py
```

You should then see the following screen:

[Image: Hello World]

If you click the button:

[Image: Hello World - Button Clicked]

NOTES

Further study materials Lesson 2

Transcript

Advanced Concepts and Features

In this section, we'll explore some advanced topics in Flet that empower you to create dynamic, responsive, and robust desktop applications.

State Management

Efficient state management is essential for ensuring that your app's UI remains in sync with its underlying data.

Flet's reactive model makes it straightforward to update UI elements when the application state changes.

In the following example, we maintain a counter state that updates a text widget each time a button is clicked:

```
import flet as ft

def main(page: ft.Page):
    counter = 0
    counter_text = ft.Text("Counter: 0")
    def increment(e):
        nonlocal counter
        counter += 1
        counter_text.value = f"Counter: {counter}"
        page.update() # Refresh the UI to show the new counter value
    inc_button = ft.ElevatedButton("Increment",
        on_click=increment)
    page.add(counter_text, inc_button)

ft.app(target=main)
```

Explanation:

- **State Variable:** A local variable `counter` holds the current count.
- **Event Handler:** The `increment` function updates this variable and the associated text widget.
- **UI Update:** Calling `page.update()` re-renders the UI to reflect changes.

Running the app and clicking the button, you will see:

[Image: Button Counter]

Responsive Design

Designing an app that adapts gracefully to different screen sizes is crucial for a good user experience.

Flet allows you to create flexible layouts that respond to window resizing.

NOTES

Further study materials Lesson 2

Transcript

In this example, the layout switches between a row and column arrangement based on the window width:

```
import flet as ft

def main(page: ft.Page):
    # Create a flexible layout with text items
    layout = ft.Row([ft.Text("Item 1"), ft.Text("Item 2"),
                     ft.Text("Item 3")])
    def update_layout(e):
        # Change layout direction based on window width
        if page.window_width < 600:
            layout.direction = "column"
        else:
            layout.direction = "row"
        page.update()
    # Set the resize handler to update the layout
    page.on_resize = update_layout
    page.add(layout)

ft.app(target=main)
```

Explanation:

- **Dynamic Direction:** The `update_layout` function checks `page.window_width` and sets the layout's `direction` property accordingly.
- **Event Binding:** The `page.on_resize` event ensures the layout adjusts whenever the window size changes.

Running the app:

[Image: Responsive Design]

Integrating APIs and Data

Modern applications often need to fetch and display data from external sources.

By leveraging Python's HTTP libraries (e.g., `requests`), you can integrate API calls within your Flet app.

The following snippet demonstrates fetching data from an API and displaying it:

```
import flet as ft
import requests

def main(page: ft.Page):
    data_text = ft.Text("Fetching data...")
    def fetch_data(e):
        try:
            response =
            requests.get("https://api.sampleapis.com/coffee/hot")
            if response.ok:
                data = response.json()
                data_text.value = data
```

NOTES

Further study materials Lesson 2

Transcript

```
else:
    data_text.value = "Failed to fetch data."
except Exception as ex:
    data_text.value = f"Error: {ex}"
page.update()
fetch_button = ft.ElevatedButton("Fetch Data",
    on_click=fetch_data)
page.add(data_text, fetch_button)
```

```
ft.app(target=main)
```

Don't forget to install the `requests` library:

```
pip install requests
```

Explanation:

- **API Request:** The `fetch_data` function makes an HTTP GET request to retrieve data.
- **Error Handling:** Proper error handling ensures the UI remains informative even when data fetching fails.
- **UI Update:** The text widget is updated with the fetched data, and the UI refreshes via `page.update()`.

Running the app and clicking the button:

[Image: Integrating APIs]

Custom Components and Themes

Creating custom components can help you maintain consistency and reusability across your app.

Flet allows you to build composite widgets and apply global themes to ensure a uniform look and feel.

Below is an example of a custom card component along with theme customization:

```
import flet as ft

# Define a custom card component by extending a Container
class CustomCard(ft.Container):
    def __init__(self, title: str, content: str, **kwargs):
        super().__init__(
            content=ft.Column([
                ft.Text(title, size=20, weight="bold"),
                ft.Text(content)
            ]),
            padding=10,
            border_radius=5,
            bgcolor=ft.colors.BLUE_100,
            **kwargs
        )

def main(page: ft.Page):
    # Apply a global theme using a color seed
```

NOTES

Further study materials Lesson 2

Transcript

```
page.theme = ft.Theme(color_scheme_seed=ft.colors.BLUE)
card1 = CustomCard("Welcome", "This is a custom card
component.")
page.add(card1)
```

```
ft.app(target=main)
```

Explanation:

- **Custom Component:** The `CustomCard` class encapsulates a title and content into a reusable card widget.
- **Theming:** A global theme is applied via `page.theme` to enforce a consistent color scheme across the application.
- **Modularization:** Custom components like `CustomCard` can be stored in separate modules for better organization and reuse.

Running the app:

[Image: Custom Components]

Debugging and Optimization

Building robust desktop applications requires not only writing functional code but also being prepared to debug and optimize as your project scales.

In this section, we cover common pitfalls developers face when working with Flet and provide best practices to optimize application performance.

Common Pitfalls

Developing with Flet can sometimes lead to issues that can be resolved with a few troubleshooting strategies:

Widget Update Issues:

- **Pitfall:** Forgetting to call `page.update()` after changing the state of widgets can result in the UI not reflecting recent changes.
- **Tip:** Always ensure that after modifying any widget property or state, you trigger an update to re-render the UI.

Event Binding Errors:

- **Pitfall:** Incorrectly binding events or misnaming callback functions may lead to unresponsive components.
- **Tip:** Double-check that your event handlers are properly assigned to UI elements and that the function signatures match what Flet expects.

Resource Handling:

- **Pitfall:** Overloading the app with too many heavy widgets or

NOTES

Further study materials Lesson 2

Transcript

not properly disposing of resources can degrade performance.

- **Tip:** Use lazy loading techniques to load heavy components only when needed, and consider breaking your UI into smaller, manageable parts.

Debugging Strategy:

- Utilize print statements or logging to trace the flow of execution.
- Test individual components in isolation before integrating them into the main application.

Performance Considerations

Optimizing performance is key to delivering a smooth user experience.

Here are some best practices for ensuring your Flet desktop app performs efficiently:

Lazy Loading:

- **Concept:** Load components or data only when they are required, rather than at the startup of the application.
- **Example:** If your app contains multiple tabs with heavy data, load the content of a tab only when the user clicks on it.

```
import flet as ft

def load_tab_content(e):
    # Simulate lazy loading of data for a tab
    content.value = "Loaded content for Tab 2"
    page.update()

def main(page: ft.Page):
    tab1 = ft.Tab(text="Tab 1", content=ft.Text("Static
content for Tab 1"))
    # Initially empty content for Tab 2
    content = ft.Text("")
    tab2 = ft.Tab(text="Tab 2", content=content)

    tabs = ft.Tabs(tabs=[tab1, tab2], on_change=lambda e:
load_tab_content(e) if e.control.selected_index == 1 else
None)
    page.add(tabs)

ft.app(target=main)
```

Running the app:

[Image: Lazy Loading]

Asynchronous Operations:

- **Concept:** Use asynchronous calls for tasks such as data fetching or heavy computations so that the UI remains

NOTES

Further study materials Lesson 2

Transcript

responsive.

- **Example:** Integrate Python's `asyncio` module to handle asynchronous tasks, ensuring long-running operations don't block the main thread.

```
import flet as ft
import asyncio

async def fetch_data_async():
    # Simulate an asynchronous API call with a delay
    await asyncio.sleep(2)
    return "Data fetched asynchronously!"

def main(page: ft.Page):
    status_text = ft.Text("Click to fetch data asynchronously.")
    async def on_click(e):
        status_text.value = "Fetching..."
        page.update()
        data = await fetch_data_async()
        status_text.value = data
        page.update()
    fetch_button = ft.ElevatedButton(text="Fetch Data",
    on_click=lambda e: asyncio.create_task(on_click(e)))
    page.add(status_text, fetch_button)

ft.app(target=main)
```

Running the app:

[Image: Asynchronous Download]

Efficient State Updates:

- Avoid unnecessary state changes and frequent calls to `page.update()` by batching updates where possible.
- Use conditional updates to ensure the UI is only refreshed when there are actual changes.

Packaging the App

Packaging your Flet app into a standalone executable ensures that end users don't need a Python environment to run your application.

One popular tool for this purpose is **PyInstaller**. Below is a basic walkthrough for packaging your app:

Install PyInstaller:

```
pip install pyinstaller
```

Create the Executable: Navigate to your project directory and run the following command:

NOTES

Further study materials Lesson 2

Transcript

```
pyinstaller --onefile main.py
```

This command generates a single executable file in the `dist/` directory. The `--onefile` flag bundles your script and its dependencies into one file.

Platform Considerations:

- **Windows:** The executable will be a `.exe` file.
- **macOS/Linux:** The output will be a binary executable that can be run directly on the target OS.

? For more complex projects, consider creating a custom spec file to include additional resources (e.g., icons, assets).

Conclusion

Throughout this article, we explored the journey of building desktop applications in Python using Flet.

We began by understanding the framework's capabilities and setting up a proper development environment.

We then walked through building a simple "Hello World" app, discussed advanced topics such as state management, responsive design, API integration, and the creation of custom components.

Finally, we covered essential debugging, optimization techniques, and practical deployment strategies to package and distribute your application.

To further enhance your skills and explore more about Flet, consider checking out the Flet Official Documentation at Flet Docs.

Embrace the power of Flet to build modern, interactive desktop applications with the simplicity of Python.

Whether you're refining your skills or venturing into new projects, continue experimenting with Flet's versatile features and discover innovative ways to solve real-world challenges.

Follow me on Twitter:

Follow me on Instagram:

Follow me on TikTok:

Follow me on YouTube:

NOTES

Further study materials Lesson 2

Content type: Text

Build a Flutter-Style Counter App with Python and the Flet Framework

Available at
<https://medium.com/better-programming/rebuilding-the-flutter-counter-app-in-python-with-flet-53d4a2e29a65>

Full
Content



Summary

Introduction to Flet and Counter App

The article explains how to rebuild the classic Flutter Counter app using Python instead of Dart. This is made possible with Flet, an open-source framework that lets programmers create Flutter applications in Python. Currently, Flet supports Python, but plans exist to expand to other languages. The Counter app has three main parts: an app bar at the top, a counter text in the middle, and a floating action button at the bottom. The goal is to show that Python can build Flutter UIs easily.

Installing Flet and Hello World

Flet is installed as a Python package using pip. It is recommended to use a virtual environment. After installation, create a Python file and run a simple 'Hello, World!' code to verify setup. The code imports Flet, defines a main function that adds a text control, and launches the app. The app can run as a native window or in a web browser by changing the view parameter. This basic example ensures the framework works before building the Counter app.

UI Configuration and App Bar

The first step in building the Counter app is setting up the page or UI. The page title is set to 'Flet Counter App', the theme mode is set to light, and Material 2 design is used to mimic Flutter. The page content is centered both horizontally and vertically. Then, an AppBar is added at the top with a white title 'Flet Demo Home Page' on a blue background. The title is centered using the `center_title` property. This creates a basic structure similar to the Flutter example.

Adding Counter Text and Floating Action Button

The middle of the app contains static text and a counter value. The `counter_text` is a Text control with initial value '0' and a display medium style. The floating action button (FAB) is placed at the bottom right. It has a round shape, blue background, white plus icon, and a tooltip 'Increment!'. The FAB's `on_click` event calls a function that increments the counter by one and updates the page. This function converts the current value to an integer, adds 1, and sets it back as a string before calling `page.update()`.

NOTES

Further study materials Lesson 2



Summary

Conclusion and Deployment Options

The resulting app closely resembles the Flutter Counter example. The author emphasizes that Flet is easy to use and a proof of concept that Python can create Flutter UIs. Users can package the app as a standalone executable or deploy it on the web, thanks to Flet's cross-platform nature. The article concludes by inviting feedback and promoting the author's YouTube channel. It encourages readers to try building apps with Flet and reach out if they encounter issues.

NOTES



Further study materials Lesson 2

Transcript

Rebuilding the Flutter Counter App in Python With Flet

A step-by-step guide to creating apps using a Python framework

[Image: Photo by Kelly Sikkema on Unsplash]

Quick Introduction

The Counter app is one of the first apps you can build when learning to create GUI applications with the famous Flutter. In this article, we will rebuild it from scratch. Not in Dart, but with Python.

To achieve this, we will use a Python framework named Flet. If you have already heard of it, keep reading. If not, let me briefly describe it: It is an open-source framework that aims to allow programmers to build Flutter applications with their favorite programming language easily.

At the time of writing, it is only available in Python, but they plan to spread to other languages such as Go and C# in the upcoming years. Give their Roadmap a quick look.

Toward the end of this article, I will conclude by giving my opinion on this Python framework.

To look at the Flutter example we will be cloning, see this article.

App's Structure

The application consists of three main parts:

- The app bar at the top
- Some text containing the counter value in the middle
- A floating action button at the right bottom

That will be all. Before we build the app, let's ensure you have Flet installed.

Installing Flet — "Hello, World!"

Flet is available as a Python package, meaning its installation is similar to that of other packages. Make sure you have Python installed, then run this command from your terminal to grab the latest version (v0.4.0 at the time of writing):

```
pip install flet
```

It is recommended to do this in a virtual environment.

After the installation, create a new file and name it as you wish (for example, `my-editor.py`), and run this basic "Hello, World!" code using your favorite IDE to ensure you are good to go.

```
import flet as ft
```

NOTES

Further study materials Lesson 2

Transcript

```
def main(page: ft.Page):  
    page.add(ft.Text(value="Hello, world!"))
```

```
ft.app(target=main)
```

[Image: Hello World - Dark Mode]

[Image: Hello world - light mode]

Throughout this article, I will be importing Flet with the alias `ft`, just like NumPy is commonly aliased `np` or Pandas `pd`. All right! Let's dive right in.

The execution of that code block opens a native window containing the rendered output. If you wish, you could tell Flet to render that output in your browser just by modifying the last line of code to:

```
ft.app(target=main, view=ft.WEB_BROWSER) # a port could  
also be specified
```

Step 1: Page/UI Configuration

Let's add some settings and configurations to our page/UI.

```
import flet as ft  
  
def main(page: ft.Page):  
    # the title of the app  
    page.title = "Flet Counter App"  
  
    # a light/bright theme  
    page.theme_mode = "light"  
  
    # use Material 2 design theme | this is just to better  
    # mimic the Flutter example  
    page.theme = ft.Theme(use_material3=False)  
  
    # center the page's content  
    page.horizontal_alignment = "center"  
    page.vertical_alignment = "center"  
  
    page.add(ft.Text(value="Hello, world!"))  
  
ft.app(target=main)
```

I added explanatory comments to ease your understanding.

Step 2: The AppBar at the Top

Now, let's add an application bar. The Flet page has an `appbar` property that only takes an `AppBar` class.

Add the code block before the `page.add()`:

```
# the app's appbar  
page.appbar = ft.AppBar(  
    title=ft.Text("Flet Demo Home Page"),
```

NOTES

Further study materials Lesson 2

Transcript

```
color=ft.colors.WHITE), # a title of white color
bgcolor=ft.colors.BLUE, # a blue background color
center_title=True # center the title; without this, the
title will be on the left
)
```

```
page.add(ft.Text(value="Hello, world!"))
```

```
ft.app(target=main)
```

Run your code, and you will see something similar to the image below:

[Image: Hello World with AppBar]

Step 3: The Counter Text in the Middle

We will now modify the text ("Hello, World!") in the middle of the application. We will add static text (a description) at the top and dynamic text (the counter value) below it.

```
# the app's appbar
page.appbar = ft.AppBar(
# ...
)

# text that contains the counter number to be incremented
counter_text = ft.Text("0",
style=ft.TextThemeStyle.DISPLAY_MEDIUM)

# adding our widgets/controls to the page/UI
page.add(
ft.Text("You have pushed the button this many times:"),
counter_text
)

ft.app(target=main)
```

Run your app, and you will see the two texts we have just added:

[Image: AppBar and Counter value]

Step 4: The Floating Action Button at the Bottom

Now we will add the floating action button (FAB) at the bottom right corner of the screen. Our FAB should have a blue background, be round/circular, and have a white plus (+) icon. You can make use of this app to choose your icons.

The FAB will also have a tooltip, the text shown when the icon is hovered over.

The Flet page has a `floating_action_button` property that only takes a `FloatingActionButton` class.

```
# the page's FAB
page.floating_action_button = ft.FloatingActionButton(
```

NOTES

Further study materials Lesson 2

Transcript

```
content=ft.Icon(ft.icons.ADD, color=ft.colors.WHITE),
shape=ft.CircleBorder(), # gives the button a
round/circle shape
on_click=increment_counter, # the callback to be executed
when this button is clicked
tooltip="Increment", # the text to be shown when this
button is hovered
bgcolor=ft.colors.BLUE # a blue background color
)

page.add(
# ...
)

ft.app(target=main)
```

In the code above, I defined a property of the FAB named `on_click` to be equal to `increment_counter`, which is the function/callback that will be called every time this button is clicked. Before running the code, we must create this callback, or Python will break and tell us it didn't find something named `increment_counter`.

This callback/function will increment the counter value by one, starting from 0. Here is its definition:

```
def increment_counter(e):
    """Increment the value of the counter_text object by 1,
    and update the UI to reflect these changes."""
    counter_text.value = str(int(counter_text.value) + 1)
    page.update()
```

Add it in the main function, either below or above the page settings we made earlier.

Run the code, and you will see the following output:

[Image: Final result]

Press on the FAB, and you will see the counter value being modified. Here is the complete code. I deployed an online version of this for you [here](#). Try it out and let me know what you think. You could eventually package this app as a standalone executable file or deploy it on the web. That's the cross-platform nature of Flet.

Do you think what we've built resembles the Flutter example? I am sure it does!

Conclusion

See this application as a proof of concept: Python alone can be used to build Flutter UIs. All thanks go to the Flet Python framework. As you've seen, it is quite easy and simple to use. I have tried several Python GUI frameworks, and personally, Flet is the easiest of all.

NOTES

Further study materials Lesson 2

Transcript

If you face any issues, let me know so I can help you out.

By the way, I started a YouTube channel, please subscribe:

“Henri Ndonko - TheEthicalBoy”

Thanks for reading.

NOTES

Further study materials Lesson 2

Content type: Text

Comparing Python GUI Frameworks for Efficient Automation

Available at
<https://medium.com/@areejkam01/i-compared-pyside6-pyqt-kivy-flet-and-dearpygui-my-honest-2025-review-8c037118a777>

Full
Content



Summary

Introduction: The Real Question for GUI Frameworks

Many students look for the best Python GUI framework for automation but get confused by conflicting claims. This review compares PySide6, PyQt, Kivy, Flet, and DearPyGUI by building real automation tools, not toy apps. The key question is which framework reduces friction between an idea and a working tool. Automation tools need to be fast to build, easy to maintain, script-friendly, and stable. The baseline task is a desktop tool that selects a folder, scans PDFs, extracts text, and renames files based on detected keywords.

PySide6 and PyQt: Qt Frameworks Compared

PySide6 is the official Qt binding for Python, offering a solid, professional feel for automation tools used over months. It features a clean signal-slot system that scales, native OS look, and LGPL license for shipping tools. The downside is Qt's large API, but you don't need to memorize everything. PyQt is extremely similar technically but has GPL/commercial licensing issues that matter when sharing or monetizing tools. For serious automation apps, PySide6 is preferred over PyQt to avoid licensing headaches.

Kivy: Powerful but Overkill for Desktop Automation

Kivy is powerful and flexible, ideal for touch interfaces, mobile deployment, and custom layouts. However, for desktop automation, it can be overkill. The layout system is verbose, desktop apps don't feel native by default, and you often write more UI code than automation logic. Kivy solves a different problem, so it's not the best choice for typical automation tasks. It may be suitable if you need cross-platform mobile or touch support, but for simple desktop tools, other frameworks are more efficient.

Flet: Shockingly Productive for Internal Tools

Flet lets you write GUI apps like web apps entirely in Python, offering ridiculously fast prototyping and a clean mental model. It is easy to extend into dashboards. For internal automation tools and quick scripts, Flet is dangerously effective. The trade-off is giving up some native desktop control, but for internal tools, that's often acceptable. Flet works well when the GUI needs to be built

NOTES

Further study materials Lesson 2



Summary

quickly and the automation logic is the priority. It surprised the author with its productivity for internal tools.

DearPyGUI and Final Advice for Automation

DearPyGUI is the fastest way to add a simple GUI to a script, with minimal boilerplate and excellent performance. It shines for script wrappers and quick internal tools where the automation matters more than the interface. However, layouts get messy fast, and it's not ideal for complex workflows. The author uses DearPyGUI or Flet for quick internal tools, PySide6 for serious automation apps, Kivy for mobile or touch, and avoids PyQt unless necessary. The final advice is that the best GUI disappears, letting Python do the work.

NOTES



Further study materials Lesson 2

Transcript

I Compared PySide6, PyQt, Kivy, Flet, and DearPyGUI, My Honest 2025 Review

Which Python GUI framework actually helps you automate real work (and which ones just look good in demos)

[Image: Photo by Fotis Fotopoulos on Unsplash]

If you've ever tried to automate a task and thought, *"This would be perfect with a small GUI"*, you've probably fallen into the same rabbit hole I did.

You Google *"best Python GUI framework"*. You open five tabs. Every framework claims to be *simple*, *powerful*, and *production-ready*.

And two hours later, you still don't know which one to use.

I've been writing Python for automation long enough to realize something important:

Most GUI comparisons are written by people building toy apps, not tools they actually use.

So instead of reviewing these frameworks based on hello-world windows or flashy screenshots, I compared them the only way that matters:

Can this framework help me automate real tasks faster?

This article is the result of building *actual automation tools* with:

- PySide6
- PyQt
- Kivy
- Flet
- DearPyGUI

Same mindset. Same problems. Same Python brain.

Let's break it down.

The Real Question Most GUI Reviews Avoid

Most beginners start with:

"Which GUI framework is the most popular?"

That's the wrong question.

The better one is:

"Which framework reduces friction between an automation idea and a working tool?"

NOTES

Further study materials Lesson 2

Transcript

Automation tools don't need fancy animations. They need to be:

- Fast to build
- Easy to maintain
- Script-friendly
- Stable when scaled beyond one button

That's the lens I used for every comparison below.

Baseline Automation Task Used for Comparison

To keep things fair, I used the **same automation task** across all frameworks:

A desktop tool that:

1. Selects a folder
2. Scans all PDFs
3. Extracts text
4. Renames files based on detected keyword

This is the kind of boring, high-value automation Python excels at.

Let's start.

PySide6

The most professional choice once you move past tutorials

PySide6 is the **official Qt binding for Python**, and it shows.

If you're building automation tools you'll actually keep using for months, PySide6 feels solid in a way few others do.

Minimal Automation UI Example

```
import sys
from PySide6.QtWidgets import QApplication, QWidget,
QPushButton, QVBoxLayout, QFileDialog

class AutomationApp(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("PDF Automation Tool")

        layout = QVBoxLayout()
        button = QPushButton("Select Folder")
        button.clicked.connect(self.select_folder)

        layout.addWidget(button)
        self.setLayout(layout)

    def select_folder(self):
        folder = QFileDialog.getExistingDirectory(self, "Select
        PDF Folder")
        print(folder)
```

NOTES

Further study materials Lesson 2

Transcript

```
app = QApplication(sys.argv)
window = AutomationApp()
window.show()
app.exec()
```

Why PySide6 Works Well for Automation

- Signal-slot system scales cleanly
- Native OS look without extra work
- Excellent long-term maintainability
- LGPL license (important if you ever ship tools)

Once your automation grows beyond one script, PySide6 stops getting in your way.

Downside: Qt's API is big. You won't memorize it, but you also don't need to.

PyQt

PySide6's twin with licensing baggage

Technically, PyQt and PySide6 are extremely similar. Practically, PyQt feels like the *older sibling that never left home*.

Here's the same automation UI in PyQt:

```
import sys
from PyQt6.QtWidgets import QApplication, QWidget,
QPushButton, QVBoxLayout, QFileDialog

class AutomationApp(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("PDF Automation Tool")

        layout = QVBoxLayout()
        button = QPushButton("Select Folder")
        button.clicked.connect(self.select_folder)

        layout.addWidget(button)
        self.setLayout(layout)

    def select_folder(self):
        folder = QFileDialog.getExistingDirectory(self, "Select
        PDF Folder")
        print(folder)

app = QApplication(sys.argv)
window = AutomationApp()
window.show()
app.exec()
```

The Real Issue Isn't Code, It's Licensing

PyQt's GPL/commercial license **matters** if you:

NOTES

Further study materials Lesson 2

Transcript

- Share tools at work
- Distribute binaries
- Monetize anything

That's why I default to PySide6 now. Less thinking. Fewer constraints.

Kivy

Powerful, flexible, and overkill for most automation tools

Kivy is impressive. It's also very easy to choose for the wrong reason.

Kivy shines when you need:

- Touch interfaces
- Mobile deployment
- Custom layouts

But for desktop automation?

Here's a simple folder selector in Kivy:

```
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.button import Button
from kivy.uix.filechooser import FileChooserIconView

class AutomationLayout(BoxLayout):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.orientation = "vertical"
        chooser = FileChooserIconView()
        self.add_widget(chooser)

class AutomationApp(App):
    def build(self):
        return AutomationLayout()

AutomationApp().run()
```

What I Learned the Hard Way

- Layout system is powerful but verbose
- Desktop apps don't feel native by default
- You write more UI code than automation logic

Kivy isn't bad. It's just solving a different problem.

Flet

Shockingly productive for internal automation tools

Flet surprised me.

It lets you write GUI apps **like web apps**, but entirely in Python.

NOTES

Further study materials Lesson 2

Transcript

Here's a working automation UI:

```
import flet as ft

def main(page: ft.Page):
    def pick_folder(e):
        print("Folder selected")

    page.add(
        ft.ElevatedButton("Select Folder", on_click=pick_folder)
    )

ft.app(target=main)
```

Why Flet Works for Automation

- Ridiculously fast prototyping
- Clean mental model
- Easy to extend into dashboards

Trade-off: You give up some native desktop control.

But for internal tools, scripts, and quick automation GUIs? Flet is dangerously effective.

"Pro tip: If a GUI takes longer to build than the automation itself, you picked the wrong framework."

DearPyGUI

The fastest way to slap a GUI on a script

DearPyGUI feels like it was built by someone tired of over-engineering.

Here's the same task:

```
from dearpygui.core import *
from dearpygui.simple import *

with window("Automation Tool"):
    add_button("Select Folder", callback=lambda:
        print("Folder selected"))

start_dearpygui()
```

Where DearPyGUI Shines

- Minimal boilerplate
- Excellent for script wrappers
- Great performance

Where It Breaks Down

- Layouts get messy fast
- Not ideal for complex workflows
- Less conventional architecture

NOTES

Further study materials Lesson 2

Transcript

I use DearPyGUI when: **The automation matters more than the interface.**

The Framework I Actually Use in 2025

Here's my honest stack:

- **Quick internal tools:** DearPyGUI or Flet
- **Serious automation apps:** PySide6
- **Mobile or touch:** Kivy
- **Avoid unless necessary:** PyQt

No framework is "best." But some are better **for automation**, and that distinction matters.

Final Advice for Automation Developers

If you remember one thing, make it this:

Your GUI should disappear into the background.

The best automation tools don't feel clever, they feel inevitable.

Build GUIs that:

- Get out of the way
- Let Python do the work
- Don't fight your future self

If you're automating something you *actually need*, the right framework will feel obvious within a day.

And if it doesn't?

That's already your answer.

NOTES

Exercises • Lesson 2

EXERCISE 1 • True or false

CONTEXT

Flet is a framework for building cross-platform applications using Python. It provides a way to create apps that run on desktop, mobile, and web platforms.

QUESTION • Indicate whether the information is true or false

Flet uses Flutter's rendering engine to draw the user interface.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 2 • Multiple choice

CONTEXT

The lesson discusses Flet's architecture and its ability to run on multiple platforms.

QUESTION • Select the correct option

How many deployment targets does Flet support?

- ☐ A 4
- ☐ B 6
- ☐ C 7
- ☐ D 5

NOTES

Exercises • Lesson 2

EXERCISE 3 • True or false

CONTEXT

Flet is a Python framework that leverages Flutter's UI capabilities to build interactive applications.

QUESTION • Indicate whether the information is true or false

Flet is a framework that can only be used to build desktop applications.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 4 • Multiple choice

CONTEXT

In the lesson, we learned about Flet and its capabilities for cross-platform development using Python.

QUESTION • Select the correct option

Which of the following platforms can Flet be used to build applications for?

- ☐ A Desktop only
- ☐ B Mobile only
- ☐ C Desktop, web, and mobile
- ☐ D Web only

NOTES

Exercises • Lesson 2

EXERCISE 5 • True or false

CONTEXT

Flet is a Python framework that allows developers to create Flutter applications using Python. It provides various properties for page customization.

QUESTION • Indicate whether the information is true or false

The 'floating_action_button' property of a Flet page accepts a string as its value.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 6 • Multiple choice

CONTEXT

When building the counter app with Flet, the `counter_text` value is updated inside the `increment_counter` function. To show the new value on screen, a specific method must be called.

QUESTION • Select the correct option

In the Flet app example, which method is called to reflect changes made to a control's value on the page?

- ☐ A `page.update()`
- ☐ B `page.rebuild()`
- ☐ C `page.render()`
- ☐ D `page.refresh()`

NOTES

Exercises • Lesson 2

EXERCISE 7 • True or false

CONTEXT

The author compared several Python GUI frameworks for automation tasks, evaluating them based on their suitability for building real tools.

QUESTION • Indicate whether the information is true or false

The author recommends avoiding PySide6 for serious automation applications.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 2

EXERCISE 8 • Multiple choice

CONTEXT

In the article, the author compares several Python GUI frameworks for building automation tools. The author provides recommendations based on real-world usage and automation needs.

QUESTION • Select the correct option

According to the author, which Python GUI framework is recommended for serious automation applications?

- ☐ A Kivy
- ☐ B PyQt
- ☐ C Flet
- ☐ D PySide6

NOTES

Exercise Answers

LESSON 2

EX.01 • True or false

True.

Flet is built on top of Flutter, which uses Skia or Impeller as its GPU renderer. This allows Flet to render modern, high-performance UIs across all platforms.

☒ I got it right ☐ I got it wrong

EX.02 • Multiple choice

Option B — “6”

Flet supports Windows, macOS, Linux, Android, iOS, and the web, totaling six platforms.

☒ I got it right ☐ I got it wrong

EX.03 • True or false

False.

Flet supports building applications for web, desktop, and mobile platforms, not just desktop.

☒ I got it right ☐ I got it wrong

EX.04 • Multiple choice

Option C — “Desktop, web, and mobile”

Flet is designed for building interactive multi-user applications across web, desktop, and mobile platforms, as stated in the lesson.

☒ I got it right ☐ I got it wrong

EX.05 • True or false

False.

According to the lesson, the 'floating_action_button' property only takes a 'FloatingActionButton' class, not a string. It must be an instance of the class.

☒ I got it right ☐ I got it wrong

EX.06 • Multiple choice

Option A — “page.update()”

In the lesson, the increment_counter function calls page.update() after modifying counter_text.value to update the UI and display the new value.

☒ I got it right ☐ I got it wrong

Exercise Answers

LESSON 2

EX.07 • True or false

False.

The author explicitly recommends PySide6 for 'serious automation apps', stating it is the most professional choice for tools you will use for months. Avoiding it contradicts the article's advice.

☒ I got it right ☐ I got it wrong

EX.08 • Multiple choice

Option D — “PySide6”

The author explicitly states that for 'serious automation apps' they use PySide6, citing its solid feel, native OS look, and LGPL license.

☒ I got it right ☐ I got it wrong

Chapter 01 • Introduction to Flet and Python Setup

Lesson 03 • Configuring Your Code Editor

69 pages • 1 hr 25 min

Lesson 03 • Configuring Your Code Editor

1 hr 25 min • 69 pages

© What you will learn

Sets up VS Code or PyCharm with Flet-friendly extensions and linting. Proper editor configuration accelerates development throughout the course.

Summary	7 min	66
Transcript	7 min	67
Further study materials		
Text • Automate Python Code Formatting with Black to End Team Style Debates	18 min	70
Text • Automate Python Code Quality with isort, Black, Flake8 and Config Files	7 min	82
Text • Master the Latest VS Code Python Features for Faster Development	10 min	88
Text • Format Python Code to PEP 8 Standards Using Black, Ruff, and Pre-commit Hooks	16 min	94
Text • Essential VS Code Extensions for Productive Python Development	17 min	104
Text • Setting Up a Python Development Environment in VS Code with Virtual Environments and Tools	8 min	113
Exercises		118
Answer key		132

Configuring Your Code Editor

Summary

Why Setup Matters

A good code editor catches errors, formats code, and connects to Python. Many beginners skip editor setup, leading to confusing errors and slow debugging. When your editor knows your Python version, understands your project structure, and formats automatically, you spend time building features instead of fixing mistakes. Spending ten minutes on configuration now pays off later.

Install Essential Extensions

Step one installs three extensions from the Extensions panel in VS Code. The Python extension provides IntelliSense and error highlighting. Pylance adds faster type checking. Black Formatter formats your code automatically. Open the Extensions panel with Ctrl+Shift+X, search for each extension from the publisher Microsoft, and click Install. Reload the window to activate them.

Select the Correct Python Interpreter

Step two tells VS Code which Python interpreter to use. Open the Command Palette with Ctrl+Shift+P, search for 'Python: Select Interpreter', and pick the one in your project's virtual environment folder (usually .venv or venv). The wrong interpreter means wrong packages and silent bugs. The correct interpreter appears in the bottom-left status bar.

Enable Black for Automatic Formatting

Step three enables Black for automatic formatting. Black is an opinionated formatter that makes every style decision for you. After setup, just save the file and Black reformats it. Open VS Code Settings with Ctrl+, search for 'format on save' and check the box. Then search for 'default formatter' and select Black Formatter for Python files. Now Ctrl+S triggers formatting.

Run Flet Scripts and Verify Setup

Step four runs Flet scripts from inside VS Code. Use the integrated terminal with Ctrl+' and type 'flet run filename.py'. The Run button is faster for quick tests but the terminal gives more control. After configuring, verify by creating a new file with a basic Flet app, check the interpreter in the status bar, save to trigger Black, and run 'flet run main.py' to see the app window.

NOTES

Configuring Your Code Editor

Transcript

Configuring VS Code for Flet Development

A good code editor does not just display static text on your screen. It catches errors before you run anything, automatically formats your code for clarity, and connects directly to your Python environment. In this lesson, we'll walk through configuring VS Code so that it supports Flet from the very moment you start typing.

Most beginners skip editor setup and pay for it later. Confusing errors and slow debugging sessions are the result. When your editor knows your Python version, understands your project structure, and formats your code automatically, you spend your time building features instead of hunting down typos and indentation mistakes. That's the payoff of spending just ten minutes on configuration now.

The Four-Step Setup Process

Alright, let me walk you through it. We'll cover four steps in order. First, we install the right extensions for VS Code. Then we select the correct Python interpreter. Next, we enable Black to format code automatically. Finally, we run Flet scripts directly from the editor—so you never have to leave VS Code. Simple enough.

Step 1: Install the Essential Extensions

Extensions turn VS Code from a bare-bones editor into a full Python and Flet development environment. Without them, you're missing linting, IntelliSense (autocompletion and code intelligence), debugging, and code formatting. So step one is straightforward: install the right set.

These three extensions give you everything needed for Python and Flet development. The Python extension from Microsoft is the foundation — IntelliSense, inline error highlighting, interpreter selection. Pylance adds faster, smarter type checking on top. And Black Formatter automatically formats your code, so you never have to think about spacing or line length. Install all three from the Extensions panel in VS Code.

First, open the Extensions panel by pressing `Ctrl+Shift+X` on your keyboard. Then search for the extension name in the search bar and make sure you pick the one from the correct publisher. For the Python extension and Black Formatter, both are published by Microsoft. After you click Install, VS Code may ask you to reload the window. Do that before moving on to ensure the extensions activate properly.

Step 2: Select the Correct Python Interpreter

After installing extensions, you need to tell VS Code which Python interpreter to use. Open the Command Palette with `Ctrl+Shift+P`, search for "Python: Select Interpreter", and pick

NOTES

Configuring Your Code Editor

Transcript

the one tied to your project. Many developers skip this and hit silent bugs—wrong interpreter means wrong packages, wrong version. Let's get it right from the start, because it directly affects debugging, linting, and formatting.

If VS Code points at the wrong Python—usually the system install—it won't find Flet. Autocomplete breaks, and imports fail without any obvious reason. Those errors only surface at runtime, which makes them harder to trace. The fix is straightforward: point VS Code at the interpreter inside your virtual environment (virtualenv), where Flet is actually installed. That one change clears up all of it.

You'll hit `Ctrl+Shift+P` to open the Command Palette. Then you type 'Python: Select Interpreter' and press Enter. VS Code will list all Python environments it can find. Look for the one inside your project's virtual environment folder, usually named `.venv` or `venv`. Select it, and you'll see the Python version appear in the bottom-left corner of the status bar. That confirmation tells you the editor is now using the right environment.

Step 3: Enable Black for Automatic Formatting

Now let's set up Black. Black is an opinionated formatter — it makes every style decision for you. No configuration, no rules to remember. You just write your code, save the file, and Black instantly reformats it. That consistency saves time and keeps your codebase looking the same, every time.

Here is what Black does in practice. On the left, code written quickly — inconsistent spacing and mixed quote styles. On the right, the exact same code after Black runs on save. Same logic, but a much cleaner structure. When you work on a Flet project with multiple files, this consistency makes reading and reviewing code significantly faster across your whole project.

It's just three settings. First, open VS Code Settings with `Ctrl+,`. Search for "format on save" and check the box. Next, search for "default formatter" and select Black Formatter for Python files. Once saved, every time you press `Ctrl+S`, Black automatically reformats your code. You never have to think about style again.

Step 4: Run Flet Scripts from the Editor

All three settings are saved. Now the final step: running Flet scripts from inside VS Code without switching windows. Two methods exist — one quick for testing, the other with more control for full launches. Knowing both lets you pick the right approach for what you're doing. You stay in the editor end to end.

The integrated terminal gives you the most control. Open it with `Ctrl+`` (backtick), then type `flet run` followed by your filename. This is the same command you would use in any terminal, and it

NOTES

Configuring Your Code Editor

Transcript

works exactly the same way. The Run button at the top right is faster for quick tests during development. One click runs the current file. For Flet development, the terminal is the better choice. You pass arguments and see full output, with full control over the environment.

Verify Your Setup

After you finish configuring, verify your setup with four quick steps. First, create a new file called `main.py` and write a basic Flet app. Second, check the status bar to confirm the right Python interpreter is selected. Third, save the file — Black will automatically format your code. Finally, open the terminal and run `flet run main.py`. If a window opens with your app, your entire setup is working correctly.

Your editor is now Flet-ready. Extensions are installed: Python, Pylance, and Black Formatter are active and ready. The interpreter is set to your virtual environment where Flet lives. And format on save is on—Black runs every time you hit save. These three pieces—extensions, interpreter, format on save—mean your environment is set. From here, every lesson goes smoother because your editor works with you, not against you.

NOTES

Further study materials Lesson 3

Content type: Text

Automate Python Code Formatting with Black to End Team Style Debates

Available at
<https://medium.com/@habbema/black-o-formatador-q-ue-acabou-com-discuss%C3%B5es-in%C3%A4teis-em-python-33fb4e67e47a>

Full
Content



Summary

The Problem of Formatting Debates

Many Python teams waste time arguing about code style. Issues like single vs. double quotes, line breaks, and spaces lead to 30–40% of pull requests having style discussions. Teams lose 2–4 hours per week on formatting, and 15–20% of commits are just formatting fixes. This drains energy and adds no value to the product. Black was created to end these useless debates by enforcing a single, automatic standard.

Black's Opinionated Philosophy

Black is an automatic formatter that makes all code look the same. It is intentionally rigid: no options or personal preferences. This removes human choice, preventing endless team discussions. Black handles line breaks, indentation, spacing, quotes, and line length (default 88). It does not lint, detect bugs, or optimize code. Its simplicity and determinism are its strength, ensuring consistency across any project.

Examples and Quick Start

Black transforms messy code into clean, consistent code. For example, a poorly formatted function with missing spaces and unclear structure becomes professional with vertical alignment and proper spacing. To start, install Black with 'pip install black', then run 'black .' to format your project. Use '--check' to verify formatting in CI. Optional config in pyproject.toml sets line length and Python version.

Integration and Automation

Black integrates with editors like VS Code, PyCharm, and Vim for auto-formatting on save. Pre-commit hooks ensure every commit is formatted. For CI/CD, add Black to GitHub Actions or GitLab CI to block unformatted code. For legacy projects, use a dedicated branch, apply Black once, commit separately, then set up pre-commit and CI. This keeps git blame clean and future diffs clear.

Overcoming Objections and Final Advice

Common objections include personal style preference, occasional

NOTES

Further study materials Lesson 3



Summary

ugly output, and perceived rigidity. But consistency matters more than personal taste. Rigidity ends discussions and saves time. Black is used by Django, FastAPI, Pandas, and more. Avoid Black only in educational code or tiny scripts. To start: install, format, commit, configure pre-commit and CI. Stop debating formatting and focus on building software.

NOTES



Further study materials Lesson 3

Transcript

[Image: https://miro.medium.com/0*MD-E2HJK0gLm1hVq.png]

Black: The Formatter That Ended Useless Discussions in Python

Complete Guide to the Formatter That Truly Standardizes Python

Introduction

If you write Python code as part of a team, you've already wasted too much time on something that shouldn't exist: **formatting debates**. Single or double quotes. Line breaks here or there. Spaces before parentheses. Identical functions written with completely different styles within the same repository.

None of this breaks the code. But all of it breaks the team's flow.

These arguments don't improve architecture, reduce bugs, or deliver user value. They merely drain energy, delay PRs, and clutter the project history with commits that change absolutely nothing from a functional standpoint.

Black was created exactly to eliminate this waste. Not to "help format better," but to **end the debate** once and for all. It imposes a single, non-negotiable, automatic standard—and it does so deliberately.

This article shows why Black works where other solutions fail, how it removes friction in code reviews, how to adopt it painlessly (even in legacy projects), and why its rigidity is not a flaw, but the main reason for its success.

If you want to stop reviewing whitespace and get back to discussing real code, keep reading.

The Pain Every Python Developer Knows

Have you ever spent 30 minutes arguing about whether to use single or double quotes in a PR? Have you seen giant commits where half the changes are just spacing adjustments? Have you felt that frustration when every file in a project follows a different style and nobody knows the "official standard"? Have you had a PR blocked because someone left a trailing blank space at the end of a line?

That's the hidden cost of inconsistent formatting. And it's bigger than you think.

The Real Cost of Manual Formatting

Let's put real numbers on the problem:

Metric	Typical Impact	PRs with style
--------	----------------	----------------

NOTES

Further study materials Lesson 3

Transcript

discussions | 30–40% | | Review time lost to formatting | 2–4 hours per week | | Commits solely for formatting | 15–20% of history | | Cross-file inconsistency | High in 80% of projects | | Code review friction | Cause of 25% of friction |

Translated: In a team of 5 devs, you lose approximately 10–20 hours per week debating something that adds no value to the product.

That's more than a full day of work wasted every week.

The Radical Solution: Black

Black is an automatic code formatter for Python that solves this problem definitively.

Its philosophy is simple and brutal:

"Any code formatted with Black looks the same."

There is no negotiation. No options. No personal preferences.

Either you use Black, or you format manually forever.

And here's the kicker: this rigidity is not a limitation—it's exactly what makes Black powerful.

Why Black Works (and Other Solutions Don't)

The Problem with Flexibility

Tools that offer "style options" seem friendly, but they create a worse problem:

- Each dev picks their own preferences
- The team wastes time deciding which options to use
- Configurations diverge between projects
- Discussions never end

Black removes human choice from the equation. And that frees you to focus on what really matters: **the logic of the code**.

The "Opinionated" Philosophy

Black is intentionally opinionated. It doesn't ask what you prefer because:

1. Your preference doesn't matter to the compiler
2. Consistency > personal style
3. Decisions have already been made for you
4. Time is finite and valuable

What Black Does (and What It Doesn't)

Black Handles:

- **Line breaks** — Decides where code should break

NOTES

Further study materials Lesson 3

Transcript

- **Indentation** — Ensures consistent 4 spaces
- **Spacing** — Operators, commas, parentheses
- **Quotes** — Standardizes to double quotes
- **Line length** — Keeps within the configured limit (default: 88)
- **Visual organization** — Breathable and readable code

All of this is **deterministic** — the same input always produces the same output.

Black Is NOT:

- **Linters** → Doesn't detect bugs or bad practices (use Ruff)
- **Semantic analyzer** → Doesn't validate logic (use Mypy)
- **Optimizer** → Doesn't improve performance (use Pytest + profiling)
- **Import manager** → Doesn't organize imports (use isort or Ruff)

Black formats. That's it. And it does it perfectly.

Examples That Show the Real Power

Example 1: Simple Function, Visual Chaos

Before Black — Works, but is a visual nightmare:

```
def process_data( data:list[int] ,flag=True):
if(flag):
return [ x*2 for x in data if x>0 ]
else:
return[ x for x in data ]
```

After Black — Professional and consistent:

```
def process_data(data: list[int], flag: bool = True):
    if flag:
        return [x * 2 for x in data if x > 0]
    return [x for x in data]
```

Example 2: Complex Function, Dramatic Impact

Before Black — Maintenance nightmare:

```
def
calculate_price(items,discout=0,tax_rate=0.1,apply_disco
unt=True,
customer_type='regular'):
if apply_discount==True and customer_type=='premium':
total=sum([item['price'] for item in items])*(1-discount)
else:total=sum([item['price'] for item in items])
return total*(1+tax_rate)
```

After Black — Readable and maintainable:

```
def calculate_price(
    items: list[dict],
    discount: float = 0,
```

NOTES

Further study materials Lesson 3

Transcript

```
tax_rate: float = 0.1,
apply_discount: bool = True,
customer_type: str = "regular",
) -> float:
if apply_discount and customer_type == "premium":
    total = sum(item["price"] for item in items) * (1 -
discount)
else:
    total = sum(item["price"] for item in items)
return total * (1 + tax_rate)
```

Clear difference:

- Arguments organized vertically
- Type hints visible
- Logic easy to follow
- Zero ambiguity

How to Get Started in 5 Minutes

1. Installation

```
pip install black
```

Verify it worked:

```
black --version
```

2. First Use

Format the entire project:

```
black .
```

Format a specific file:

```
black app.py
```

See what would change (without applying):

```
black . --diff
```

Check if already formatted (ideal for CI):

```
black . --check
```

3. Minimal Configuration

Create `pyproject.toml` at the project root:

```
[tool.black]
line-length = 88
target-version = ['py311']
```

Why 88 characters? It's not random. It's the ideal balance between:

- Readability (not too wide)
- Screen utilization (not too narrow)

NOTES

Further study materials Lesson 3

Transcript

- Clean diffs (fewer unnecessary breaks)

Integration with Your Editor

VS Code

File `.vscode/settings.json`:

```
{
  "[python]": {
    "editor.formatOnSave": true,
    "editor.defaultFormatter": "ms-python.black-formatter"
  }
}
```

Install the extension: `ms-python.black-formatter`

PyCharm

1. Settings → Tools → Black
2. Check "On code reformat"
3. Check "On save"

Vim/Neovim

With `null-ls.nvim`:

```
lua
local null_ls = require("null-ls")
null_ls.setup({
  sources = {
    null_ls.builtins.formatting.black,
  },
})
```

Full Automation: Pre-commit Hooks

Why Use Pre-commit?

Ensures that **every commit is already formatted**, without exception.

Configuration

File `.pre-commit-config.yaml`:

```
repos:
- repo: https://github.com/psf/black
  rev: 24.10.0
  hooks:
  - id: black
```

Installation:

```
pip install pre-commit
pre-commit install
```

Done. From now on:

- Every `git commit` runs Black automatically

NOTES

Further study materials Lesson 3

Transcript

- Poorly formatted code does not enter the repository
- Zero forgetting, zero exceptions

CI/CD: Ensuring Quality

GitHub Actions

File `.github/workflows/quality.yml`:

```
name: Code Quality

on: [push, pull_request]
jobs:
  quality:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install black ruff

      - name: Check formatting with Black
        run: black . --check --diff

      - name: Lint with Ruff
        run: ruff check .
```

If the code is not formatted:

- Pipeline fails
- PR is blocked
- Dev fixes before merging

Technical debt resolved before it becomes a problem.

GitLab CI

File `.gitlab-ci.yml`:

yaml

```
quality:
  image: python:3.11
  script:
    - pip install black ruff
    - black . --check
    - ruff check .
```

The Complete Quality Stack

Black does not work alone. It is part of an ecosystem:

NOTES

Further study materials Lesson 3

Transcript

Full Quality Command

```
# Format code
black .

# Organize imports and lint
ruff check . --fix
# Validate types
mypy .
# Run tests
pytest --cov
```

Applying Black to Legacy Projects

Safe Strategy in 4 Steps

1. Create an exclusive branch

```
git checkout -b chore/apply-black
```

2. Run Black once

```
black .
```

3. Isolated formatting commit

```
git add .
git commit -m "chore: apply black formatting to entire
codebase"
```

4. Configure pre-commit and CI

```
# Pre-commit
pip install pre-commit
pre-commit install

# Add check in CI (see previous section)
```

Why an Isolated Commit?

- **Git blame remains useful** – formatting does not pollute history
- **Review is trivial** – only formatting, no logic
- **Future diffs stay clean** – no style noise

Common Objections (and Why They Are Wrong)

"But I prefer my personal style"

Answer: Your style doesn't matter. What matters is team readability.

Code is read 10 times more than it is written.
Consistency > personal preference.

"Black makes the code ugly in some cases"

Answer: "Ugly" is subjective. "Consistent" is objective.

You may disagree with a Black choice, but you will

NOTES

Further study materials Lesson 3

Transcript

always know what to expect.

"What if I disagree with a specific rule?"

Answer: You will disagree. Everyone disagrees with something.

But you accept it and move on – **that's exactly the point.**

"My editor already formats automatically"

Answer: Each editor formats differently. Black ensures everyone formats the same.

Without Black:

- Dev 1 uses VS Code → format A
- Dev 2 uses PyCharm → format B
- Dev 3 uses Vim → format C

With Black: everyone uses the same standard, regardless of editor.

"Black is too rigid"

Answer: Rigidity is a feature, not a bug.

Flexibility generates:

- Endless discussions
- Divergent configurations
- Wasted time
- Inconsistent code

Rigidity generates:

- Zero discussion
- Zero configuration
- Zero wasted time
- Uniform code

When NOT to Use Black

Rare cases where Black may not be ideal:

- 1. Educational code** where style is part of the pedagogical explanation
- 2. Tiny projects** (1–2 files, disposable script)
- 3. Code that must follow external mandatory standards** (regulatory, legacy system)

Outside these, there is no real technical reason to avoid Black.

Real-World Use Cases

Projects and companies that use Black:

- **Django** – Most popular Python web framework
- **FastAPI** – Modern API framework

NOTES

Further study materials Lesson 3

Transcript

- **Pandas** – Data analysis library
- **Pytest** – Testing framework
- **Poetry** – Dependency manager

If these projects trust Black, so can you.

Useful Commands – Quick Reference

```
# Format everything
black .

# Format specific file
black app.py
# See differences without applying
black . --diff
# Check without modifying (CI)
black . --check
# Exclude directories
black . --exclude "/(\\.git|\\.venv|build)/"
# Format with specific line length
black . --line-length 100
# See only files that would be modified
black . --check --diff
```

Implementation Checklist

Use this list to implement Black in your project:

Initial Setup

- Install Black: `pip install black`
- Run first time: `black .`
- Isolated commit: `git commit -m "chore: apply black"`
- Create `pyproject.toml` with configuration

Local Automation

- Install pre-commit: `pip install pre-commit`
- Create `.pre-commit-config.yaml`
- Activate hooks: `pre-commit install`
- Configure editor (VS Code, PyCharm, etc.)

Remote Automation

- Add Black check in CI/CD
- Configure pipeline to block poorly formatted PRs
- Document in the project README

Team Communication

- Explain the change to the team
- Show benefits (less review, more focus)
- Ensure everyone configured editor + pre-commit

Conclusion: Stop Debating, Start Building

NOTES

Further study materials Lesson 3

Transcript

Black solves a problem that shouldn't exist, yet consumes precious time in almost every Python project: **inconsistent formatting and useless style discussions**.

It doesn't ask for your opinion. It doesn't negotiate your preferences. It simply formats.

And by doing so, it frees your team to focus on what truly matters: **building software that solves real problems**.

Immediate Action for Monday

Install Black:

```
pip install black
```

Format your project:

```
black .
```

Commit the change:

```
git add .  
git commit -m "chore: apply black formatting"
```

Configure pre-commit:

```
pip install pre-commit  
pre-commit install
```

Add check in CI (copy example from CI/CD section)

Total time: 10 minutes. Benefit: forever.

Stop reviewing whitespace. Stop arguing about quotes. Stop wasting time on formatting.

Use Black and get back to building software that matters.

See you next time!

NOTES

Further study materials Lesson 3

Content type: Text

Automate Python Code Quality with isort, Black, Flake8 and Config Files

Available at
<https://medium.com/@naveenasrit/python-code-formatting-with-isort-black-flake8-0a296fe7fe4f>

Full
Content

Summary

Python Code Formatting Tools Overview

When working on a Python project in a team, code can become messy because different developers format code differently. Tools like `isort`, `Black`, and `Flake8` help keep code clean, consistent, and error-free. `isort` organizes import statements, `Black` reformats code to a single style, and `Flake8` checks for mistakes and style issues. Using these tools together makes the codebase professional and team-friendly.

isort - Sorting Imports

isort stands for import sorter. It automatically organizes import statements by grouping standard library, third-party, and local imports. It also removes duplicates and sorts them alphabetically. For example, before using isort, imports like 'import pandas as pd' and 'import os' may be mixed. After isort, standard library imports come first, then third-party, making the code cleaner and easier to read.

Black - The Uncompromising Formatter

Black is an uncompromising Python code formatter that enforces a consistent style. It automatically reformats code, removing debates about spaces, tabs, or line breaks. Developers just write code, and Black formats it. For instance, a function defined as `'def add(x,y): return x+ y'` becomes `'def add(x, y): return x + y'` after Black. This ensures the entire codebase looks uniform.

Flake8 - Code Checking and Automation

Flake8 is a linter that points out mistakes and style issues in Python code. It finds unused imports, unused variables, warns about lines that are too long, and ensures PEP 8 guidelines are followed. Example error messages include 'E231: missing whitespace after ',' and 'F401: 'os' imported but unused'. To automate formatting, install the tools with 'pip install isort black flake8' and use pre-commit hooks to run them before each Git commit.

Configuration File Formats in Python

Python projects use configuration files to manage settings. TOML is the new standard, used by tools like Black and isort in

NOTES

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Further study materials Lesson 3



Summary

'pyproject.toml'. JSON is lightweight and used for data exchange, but does not support comments. YAML is human-readable and supports comments, often used in DevOps. INI is older and used for simple app configurations. Choose TOML for Python project settings, YAML for deployment, JSON for APIs, and INI for local configs.

NOTES



Further study materials Lesson 3

Transcript

Python Code Formatting with isort, Black, Flake8

When working on a Python project, especially in a team, code can quickly become messy. Different developers may format code in different ways. That's where tools like **isort**, **Black**, and **Flake8** come in. They help keep your code **clean, consistent, and error-free**.

[Image: https://miro.medium.com/1*HL45PQvDLHqIjrjffD0GVQ.png]

isort - Sorting Imports

isort stands for *import sorter*.

- It automatically organizes your `import` statements.
- Groups standard library, third-party, and local imports.
- Removes duplicates and sorts alphabetically.

Before isort:

```
import pandas as pd
import os
import sys
import numpy as np
```

After isort:

```
import os
import sys

import numpy as np
import pandas as pd
```

Black - The Uncompromising Formatter

Black is a Python code formatter that enforces one consistent style.

- It automatically reformats your code.
- Removes arguments about "spaces vs tabs" or "where to break lines".
- Developers just write code → Black formats it.

Before Black:

```
def add(x,y): return x+ y
```

After Black:

```
def add(x, y):
    return x + y
```

Flake8 - The Code Checker

Flake8 is a linter that points out mistakes and style issues.

- Finds unused imports, unused variables.

NOTES

Further study materials Lesson 3

Transcript

- Warns about lines that are too long.
- Ensures you follow PEP 8 style guidelines.

Example error messages from `flake8`:

- E231: missing whitespace after ',','
- F401: 'os' imported but unused

Installation commands

```
pip install isort black flake8
```

Automating with pre-commit (optional but valuable) automates these tools to run before each commit.

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/pycqa/isort
  rev: 5.12.0
  hooks:
  - id: isort
  - repo: https://github.com/psf/black
    rev: 23.3.0
    hooks:
    - id: black
  - repo: https://github.com/pycqa/flake8
    rev: 6.0.0
    hooks:
    - id: flake8
```

Then activate with:

```
pre-commit install
```

This ensures formatting happens automatically before every Git commit.

Workflow Together

1. Run `isort` → Clean up imports.
2. Run `black` → Format everything.
3. Run `flake8` → Check for errors/warnings.

File Formatting in Python

Apart from code, projects often need **configuration files**. Python has multiple formats for this:

TOML (`pyproject.toml`)

- TOML is the new standard for Python project configuration.
- Tools like **Black**, **isort**, and **flake8** can read settings from it.

`pyproject.toml`:

```
[tool.black]
line-length = 88
```

NOTES

Further study materials Lesson 3

Transcript

```
[tool.isort]
profile = "black"
```

```
[tool.flake8]
max-line-length = 88
```

Used by: Black, isort, Flake8, Poetry, Hatch, PDM, etc. Easy to read and supports nested tables (sections).

JSON (.json)

JSON (JavaScript Object Notation) is a lightweight, text-based format used to store structured data. Python has built-in support for it with the `json` module.

```
{
  "name": "spellspark",
  "version": "1.0",
  "dependencies": ["numpy", "pandas"]
}
```

Used by: APIs, web apps, data exchange Easy for machines and humans to read Doesn't support comments

YAML (.yaml / .yml)

YAML (YAML Ain't Markup Language) is often used for configuration files - more readable than JSON. It supports comments and complex data like lists and dictionaries.

```
project:
  name: my_project
  version: 1.0

dependencies:
  - numpy
  - pandas
```

Used by: Docker, Kubernetes, GitHub Actions, CI/CD tools
Human-friendly syntax Be careful with indentation - it matters!

INI (.ini) using ConfigParser

INI files (also known as config files) are older but still used for simple app configurations.

```
[database]
host = localhost
port = 5432
user = admin
password = secret
```

- Use **TOML** for Python project settings (`pyproject.toml`).
- Use **YAML** for DevOps or deployment configs.
- Use **JSON** for data exchange or APIs.
- Use **INI** for lightweight local configurations.

NOTES

Further study materials Lesson 3

Transcript

Using tools like **isort**, **Black**, and **Flake8** keeps your Python code clean, consistent, and error-free. And by using configuration files like **TOML**, **YAML**, **JSON**, or **INI**, you can easily manage your project settings in an organized way. Together, they make your codebase more professional, readable, and team-friendly.

NOTES

Further study materials Lesson 3

Content type: Text

Master the Latest VS Code Python Features for Faster Development

Available at
<https://towardsdev.com/what-the-new-vs-code-python-extension-brings-in-june-2025-bc3af7da4c11>

Full
Content



Summary

Integrated Python Chat Assistant

The June 2025 update introduces a built-in Python chat assistant accessible via the command palette. It provides context-aware help by connecting to the active workspace. Users can ask for code explanations, refactorings, or debugging suggestions. The assistant uses static analysis and language server hints to give solid advice, saving time and helping junior developers spot issues.

REPL with Language Server Completions

The terminal REPL now offers completions powered by the same IntelliSense engine used in the editor. When typing in a Python session, suggestions appear with docstrings and argument hints. This removes the gap between the console and editor, making the REPL more useful for quick calculations and debugging. The feature is fast and dynamic, and can be disabled in settings.

Create Project from Template

A new command 'Create Project from Template' simplifies project setup. It offers templates for basic scripts, Flask, FastAPI, data science, and pytest. The extension creates folders, .gitignore, requirements.txt, and optionally pyproject.toml. Users choose the environment tool (venv, conda, Poetry) during setup. This reduces decision fatigue and helps teams standardize.

Pyenv and Poetry Environment Detection

The environment selector now automatically detects pyenv and Poetry environments. It recognizes .python-version files and .venv folders. Users can switch between system Python, pyenv virtualenvs, Poetry environments, and conda environments, each tagged with the tool name. This brings full parity across environment managers without manual configuration or plugins.

Why These Updates Matter

These features reduce context switching, bridge script writing and interactive exploration, give new developers a strong foundation, and help teams ship consistent results. The update makes Python in VS Code more modern and robust, closer to heavier IDEs like PyCharm without bulk. It installs cleanly without breaking existing

NOTES

Further study materials Lesson 3



Summary

projects, and more info is available in the release notes.

NOTES

Further study materials Lesson 3

Transcript

What the New VS Code Python Extension Brings in June 2025

[Image: What the New VS Code Python Extension Brings in June 2025]

The Python development experience in Visual Studio Code has taken another step forward. On June 16, 2025, Microsoft released the latest update to the Python extension for VS Code. This release introduces several changes that simplify workflow, improve interactivity, and tighten integration with popular environment tools.

If you use Python regularly in VS Code, these updates will likely shift how you interact with your environment. The focus is on speed, clarity, and better project structure from the start.

Integrated Python Chat Tools

The most talked-about feature in this release is the built-in Python chat assistant. This is not just a chatbot that tells you how to write for loops. It connects to your active workspace and offers context-aware help. You can ask for code explanations, refactorings, or even project-specific debugging suggestions.

This tool is accessible through the command palette. Hit `Ctrl+Shift+P` and search for "Python: Chat Assistant". A new sidebar will appear, allowing you to type questions or select code in the editor and ask for details.

For example, selecting this function:

```
def fetch_users():  
    response = requests.get("https://example.com/api/users")  
    return response.json()
```

Then asking, "Can this raise any exceptions?" would yield a structured reply covering network errors, `JSONDecodeError`, and how to handle them with `try` blocks. The assistant draws from static analysis, language server hints, and broader context to give solid advice.

This makes the assistant more than just a gimmick. It saves time in research and helps junior developers spot issues they might miss.

Language Server Suggestions in the Terminal

Another big shift is REPL support backed by the Python Language Server. When you open a terminal in VS Code and launch a Python session using `python`, the REPL will now suggest completions powered by the same IntelliSense engine used in the editor.

NOTES

Further study materials Lesson 3

Transcript

Type part of a variable or function name and hit `Tab`. You will get completions with docstrings and argument hints.

```
>>> impo
import # suggested
>>> import mat
math # suggested
```

This feature removes a long-standing gap between the scripting console and the main editing window. It turns the REPL into a more useful tool, especially for quick calculations, one-off scripts, or debugging imported modules.

The suggestions are fast. They do not lag like earlier prototypes. They also update dynamically as you define new variables or import additional modules.

If you want to disable it, add the following setting to your `settings.json`:

```
"python.terminal.languageServerCompletions": false
```

Create Project from Template

Project setup has always been a sticking point for beginners. This release introduces a new command: "Create Project from Template." You can launch it from the command palette or through the Python sidebar.

The command prompts you to choose a template from a list. Options include:

- Basic script project (main.py and README.md)
- Flask web app
- FastAPI project with uvicorn runner
- Data science project with Jupyter, pandas, and matplotlib
- Pytest-based test structure

Once selected, the template initializes with folders, `.gitignore`, `requirements.txt`, and optionally `pyproject.toml`. You choose the environment tool during setup—`venv`, `conda`, or `Poetry`.

For example, if you choose the Flask template, the resulting structure looks like this:

```
my-flask-app/
├── app/
│   ├── __init__.py
│   └── routes.py
├── requirements.txt
└── .gitignore
```

NOTES

Further study materials Lesson 3

Transcript

└─ run.py

The extension does not rely on cookiecutter under the hood. Instead, it uses Microsoft-curated templates updated monthly and stored at:

This reduces decision fatigue. It also helps teams standardize on a structure that works across different machines.

PyEnv and Poetry Integration in the Environment Panel

Until now, managing environments in VS Code worked best with `venv` or `conda`. Other tools like `pyenv` or `Poetry` needed manual tweaks to be fully recognized. The June update changes that.

The Python environment selector now picks up environments created by `pyenv` and `Poetry` automatically. You do not need to modify `settings.json` or point VS Code to the interpreter path. The extension detects the `.python-version` file (used by `pyenv`) and `.venv` folders created by `Poetry`.

When you open a workspace, you'll see a new dropdown at the bottom left of the window. Click it, and you can switch between:

- System Python
- `pyenv` virtualenvs
- Poetry-managed virtualenvs
- Conda environments (if installed)

Each option is tagged with its tool name for clarity:

```
? Python 3.11.3 (pyenv)
? Python 3.10.9 (Poetry)
? Python 3.8.10 (conda)
```

If you're working on multiple projects, this makes isolation effortless. It also improves support for `.python-version` files in teams that rely on `pyenv` for version pinning.

Here's what the updated `settings.json` might look like after selecting a Poetry environment:

```
{
  "python.defaultInterpreterPath":
    "/Users/you/.cache/pypoetry/virtualenvs/my-project-py3.11/bin/python"
}
```

This change brings full parity across Python environment managers. There is no longer a need for plugins or terminal hacks to make them work.

NOTES

Further study materials Lesson 3

Transcript

Why It Matters

Each of these features serves a clear goal. The chat assistant reduces context switching. The improved REPL bridges the gap between script writing and interactive exploration. Template-based project creation gives new developers a strong foundation. And better environment management helps teams ship consistent results.

These improvements make Python in VS Code more modern and robust. They push it closer to the experience provided by heavier IDEs like PyCharm, without the bulk.

More importantly, they come without breaking existing projects. The update installs cleanly and does not change interpreter paths or override older settings.

To learn more, check the release notes at:

And for issue tracking or feedback, visit:

Thank you for reading this article. I hope you found it helpful and informative. If you have any questions, or if you would like to suggest new Python code examples or topics for future tutorials, please feel free to reach out. Your feedback and suggestions are always welcome!

Happy coding! Py-Core.com Python Programming

NOTES

Further study materials Lesson 3

Content type: Text

Format Python Code to PEP 8 Standards Using Black, Ruff, and Pre-commit Hooks

Available at
<https://python.plainenglish.io/python-code-formatter-black-or-ruff-b88f8260cf1b>

Full
Content



Summary

Why Well-Formatted Code Matters

Well-formatted code is important because it shows respect for the reader and makes code easier to understand. Python has guidelines called PEP 8 that tell us how to write clean code, like using 4 spaces for indentation and proper comments. Code formatters help automate this so we don't have to remember every rule.

Python Code Formatting Rules

PEP 8 is the style guide for Python. It covers indentation, comments, naming, and layout. For example, block comments explain the next code, and inline comments should be two spaces after the code. Following these rules makes projects last longer. Code formatters apply these rules automatically so you can focus on logic.

Using Black Code Formatter

Black is a popular Python formatter that follows PEP 8. You install it with 'pip install black' and run 'black .' to format all files. It handles vertical and horizontal formatting, like breaking long lines or removing extra spaces. It tries to fit code into 88 characters per line. Black is the default for many teams because it is reliable and easy to use.

Ruff: A Fast Alternative

Ruff is a new formatter written in Rust, making it very fast (30x faster than Black). It works as a drop-in replacement for Black and can use single quotes instead of double quotes. You install with 'pip install ruff' and run 'ruff format' to format code. It also checks for errors like unused imports. You can configure it with a ruff.toml file and set up pre-commit hooks to run automatically.

Choosing a Formatter

Formatting code is a good habit because we code for others, not just ourselves. Black has been used for many years and works well. Ruff is newer and interesting due to its speed and options. The choice depends on project needs. Using a formatter with pre-commit helps keep code clean from the start. Both tools are

NOTES

Further study materials Lesson 3



Summary

excellent for maintaining Python code quality.

NOTES

[illegible]

Further study materials Lesson 3

Transcript

Python Code Formatter - Black or Ruff?

Understand how code formatting works in Python

[Image: Photo by Max Kleinen on Unsplash]

Not a member yet? Read it for free here!

If there's one thing I like, it is well-formatted code. It's like staring at a work of art.

Well-formatted code says a lot about the person who coded it. When I see such code, I automatically admire and respect the programmer(s) responsible for it.

On the other hand, when I see poorly formatted code, I cannot help but feel weird and wonder how someone can format code and leave it like that.

I mean, "As long as it's working, who cares?", right?

Wrong.

Thankfully, we can format our Python code without much effort. There are plenty of tools out there that can do this job for us.

We will cover two popular Python code formatters that you can choose from to help you maintain your code in order.

By the end of this article, you will be able to:

- Understand why it is important to keep your code well-formatted
- Set up and use a Python formatter
- Choose and apply the one you like the most in your project

Code Formatter

Formatting code is a way of making sure that your code is well organized, readable, and that it follows the rules and guidelines of a given programming language.

As Guido said, "The code is read much more often than it is written." So making sure our code is *pleasant* to read is our duty.

In Python, we have PEP 8 as the Style Guide for Python code. If we want our code to adhere to the Python standard, we must follow it.

It provides guides on code layout, comments, naming conventions, and more.

In Python, we know that our code needs to be indented with 4 spaces per level:

```
def foo():
```

NOTES

Further study materials Lesson 3

Transcript

```
# 4 spaces from the line start ?  
print("4 spaces indentation level")
```

```
def bar():  
    # 3 spaces from the line start ?  
    print("3 spaces indentation level. This won't work!")
```

We have rules and guidelines on how to better comment on our code using one of these possible comments:

— Block comments

When we want to comment on the line of code that follows or everything within a scope:

```
def foo():  
    # Block comment about what this function is for  
    print("4 spaces indentation level")
```

— Inline Comments

Inline comments are comments that are at the end of a statement. The comment guideline for this says that it should be separated from the statement with two spaces:

```
def foo():  
    print("4 spaces indentation level") # inline comment.  
    print a message
```

Each of these guidelines that we covered here and the rest that we didn't cover, if well applied, can make a difference in our project longevity.

The good thing is that you don't need to worry about reading PEP 8 and trying to follow it line by line.

That's why we have *code formatters*. These are tools that automate this process for us in a convenient way.

In this article, we are going to talk about two major Python formatters - the OG of formatters (Black) and the new kid in town (Ruff).

So let's see more about them.

Black Code Formatter

Black is considered "the OG of Python formatters."

Black is the default formatter used by our team in every project we create. It helps us speed up the formatting so we can focus only on making sure that our code runs as expected.

It is PEP 8 compliant, which means that it follows the guidelines defined by Python.

NOTES

Further study materials Lesson 3

Transcript

Before we delve into it in more detail, let's make sure you have it installed in your virtual environment:

```
pip install black # run this command to install black
```

Now let's look at one example of poorly formatted code:

```
from my_lib import (
    module_a,
    module_b,
    module_c
)
```

```
def foo():
```

```
    print("Unnecessary space above this print.")
```

To automatically format our code, we can run the *black* command. There are two ways of doing that: a) by running *black* . which formats everything inside the current directory, or b) by running *black your_module.py* to format a single Python file.

```
black . # format everything inside the current directory
```

```
from my_lib import module_a, module_b, module_c # make
the import fit one line
```

```
def foo():
```

```
    print("Unnecessary space above this print.") # remove
extra white space leaving only one
```

Now let's understand how Black manages code formatting horizontally and vertically, and how it decides how to format a line of code.

To format a horizontal line of code, Black does whatever needs to be done to comply with *pycodestyle*.

If you don't know, *pycodestyle* is simply a tool that you can install and run to check if your code follows PEP 8 conventions.

```
pip install pycodestyle # you can install it to run a
test on our example
```

Let's run *pycodestyle* on the following code snippet and see what we get.

```
# foo.py
def foo():
```

```
    print("Unnecessary space above this print.")
```

NOTES

Further study materials Lesson 3

Transcript

As you can see, we have too many whitespaces between the function definition statement and the *print* statement, which is against the PEP 8 convention.

```
pycodestyle --first foo.py # run pycodestyle against our
module foo.py
:> foo.py:4:5: E303 too many blank lines (2) # output
```

— Vertical formatting

Vertical formatting is applied to multiple imports, function definitions, function calls, lists, tuples, dict definitions, etc.

First, *black* tries to format/render a full expression per line. If it's too long for the line, *black* breaks the line and puts the contents inside the brackets or parentheses on a new line.

```
my_list = [
0,
1,
2,
3,
4,
5,
6,
7,
8,
9,
] # First, black tries to put this in a single line
```

```
my_list_formatted = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

If it is still too long for the line (which by default is 88 characters per line), then it breaks the line and puts each item on a single line separated by a comma.

```
# If the contents are still too long in a single line,
break one line
my_list = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13,
14, 15, 16, 17, 18, 19, 20]

# break line starting at the first bracket
my_list_formatted = [
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20
]
```

Another example would be with a function definition where we have multiple parameters:

```
# unformatted code
def foo[T](name: str, age: int, email: str, birth_date:
str, country: str, city: str, gender: str) -> T:
# do something
...
```

NOTES

Further study materials Lesson 3

Transcript

```
# black formatted code
def foo[T](
  name: str,
  age: int,
  email: str,
  birth_date: str,
  country: str,
  city: str,
  gender: str,
) -> T:
```

All this can be simply done by running the *black* command like we saw before.

Given Black's popularity as the OG of Python formatters, you might wonder why we should explore other alternatives.

So let's take a look at another color — purple.

Ruff Code Formatter

A Python code formatter written in... Rust!

Because Ruff is written in Rust, it is faster than Black. I mean, really fast — 30x faster.

[Image: Screenshot from Ruff's site taken by the Author]

If you have a large project that you never formatted and you decide that now might be a good time to format it to be compliant with PEP 8, Ruff's performance might be very handy.

If you have been using Black in your project as the default code formatter, no problem. *Ruff* is compatible with it. It was designed to be a "drop-in replacement" for *Black*.

One thing that I like a lot about Ruff is that it is possible to configure the desired quote style (single quote or double quote). Black uses double quotes and so far does not have any configuration to switch to single quotes.

```
# Black formatter
name = "Jane Doe"

# Ruff formatter
name = 'Jane Doe'
```

Despite being super new to the market, Ruff has been gaining popularity among Python programmers and now has over 27K stars on GitHub.

So let's give it a try and see how it works:

```
pip install ruff # install ruff in your virtual
environment
```

NOTES

Further study materials Lesson 3

Transcript

Bear in mind that Ruff is only compatible with Python 3.7+. So if you're using a lower version of Python, make sure you upgrade it.

Let's try Ruff on this code:

```
# foo.py
import os, sys

my_list = [
0,
1,
2,
3,
4,
5,
6,
7,
8,
9,
]

def foo():

    print("Unnecessary space above this print.")
```

The first thing we are going to try is to run it as we did with *pycodestyle* to see what is wrong with our code:

```
ruff check # lint all files in the current directory

# output
foo.py:1:1: E401 [*] Multiple imports on one line
foo.py:1:8: F401 [*] `os` imported but unused
foo.py:1:12: F401 [*] `sys` imported but unused
Found 3 errors.
```

If we want to check only a specific file or a specific directory, we can do it by adding the *path* in the command:

```
ruff check path/to/code # run ruff in a specific
directory
ruff check foo.py # run ruff only on foo.py module
```

To format our code using Ruff, we run the following command:

```
ruff format

import os, sys

my_list = [
0,
1,
2,
3,
4,
5,
6,
```

NOTES

Further study materials Lesson 3

Transcript

```
7,  
8,  
9,  
]
```

```
def foo():  
    print("Unnecessary space above this print.")
```

See that it complained about multiple imports on one line — *import os, sys*. In this case, both Black and Ruff can do nothing about this kind of situation (yet).

If you want to configure your formatter to ignore certain files or directories, to use a single quote instead of a double, the line length to consider, etc., you can do it inside a *ruff.toml* file placed in your project root directory.

```
# ruff.toml  
exclude = ["venv"] # everything you wish to ignore,  
                  similar to gitignore  
  
line-length = 88  
indent-width = 4  
  
[format]  
quote-style = "single" # Use a single quote instead of  
double
```

If you want to take things to the next level, you can configure Ruff to check your code every time you commit using it with the *pre-commit* framework.

First, install *pre-commit* and then we will configure it:

```
pip install pre-commit
```

Create a *.pre-commit-config.yaml* file in your project root directory and paste the following configuration:

```
# .pre-commit-config.yaml  
repos:  
- repo: https://github.com/astreal-sh/ruff-pre-commit  
  rev: v0.4.4  
hooks:  
- id: ruff-format
```

You could also do the same thing for Black in case you decide to stick with it:

```
# .pre-commit-config.yaml  
repos:  
- repo: https://github.com/psf/black  
  rev: v24.4.2  
hooks:  
- id: black
```

NOTES

Further study materials Lesson 3

Transcript

Final Thoughts

Formatting your code is not only a good idea, it is a good habit that every programmer should adopt. Remember, we don't code for ourselves.

I've been using Black for almost 3 to 4 years and I like it. It does the job pretty well. But I have to admit that Ruff seems interesting and I might discuss it with my team and try to use it in some of our projects.

Do you use any formatting tool in your Python projects? If so, which is your favorite?

Hi! Thank you for your time reading my article. If you enjoyed this article and would like to receive similar content directly to your inbox.

"Join me on this journey to discover Python and more about programming"

In Plain English ?

*Thank you for being a part of the **In Plain English** community! Before you go:*

- Be sure to **clap** and **follow** the writer ?? ??
- Follow us: **X** | **LinkedIn** | **YouTube** | **Discord** | **Newsletter**
- Visit our other platforms: **Stackademic** | **CoFeed** | **Venture** | **Cubed**
- Tired of blogging platforms that force you to deal with algorithmic content? Try **Differ**
- More content at **PlainEnglish.io**

NOTES

Further study materials Lesson 3

Content type: Text

Essential VS Code Extensions for Productive Python Development

Available at
<https://medium.com/@minghunghsieh/vsc-visual-studio-code-101-5-10-python-extension-in-recommendation-a87d295c7241>

Full
Content



Summary

Introduction and Overview of VS Code Python Extensions

Visual Studio Code is a free IDE with a rich Microsoft marketplace. This story shares the best Python extensions from over 100 tested. These can boost productivity by 30%. The recommendation starts with the Python Extension Pack. The article covers four categories: Python Language Pack, Jupyter, Django, and Project/Document generators. The introduction shows how to use VS Code for Python editing, running, and debugging.

Python Extension Pack and Key Components

The Python Extension Pack by Don Jayamanne includes popular extensions: Python, Pylance, Jupyter, Jinja, Django, Python Environment Manager, Python Docstring Generator, and Python Indent. Python adds IntelliSense, linting, debugging. Pylance is a language server. Snippets provide code templates. Environment Manager shows virtual environments. Indent fixes indentation. Type Hint adds type hints. Test Explorer runs tests. All are highly recommended for Python development.

Jupyter Extensions for Interactive Programming

Jupyter extension provides notebook support with IntelliSense and debugging. It is useful for prototyping Python code. Jupyter Keymap matches the keybindings of Jupyter Notebook. Jupyter Notebook Renderers allows display of images like gif, png, svg, and plots. These extensions make VS Code a great alternative to Jupyter Lab. The article recommends trying them to see which IDE you prefer. They come as part of the Python pack or can be installed separately.

Django and Jinja Extensions for Web Development

Django extension provides beautiful syntax and snippets for Django projects. Djaneiro offers a richer collection of snippets for templates, models, views, and forms. Jinja adds language support for Jinja templates, providing colorization. These extensions help speed up Django web development. The article rates Django extension 5 stars and Djaneiro 4 stars. They are essential for perfectionists with deadlines.

NOTES

Further study materials Lesson 3



Summary

Project and Document Generators and Final Summary

autoDocstring quickly generates docstrings for Python functions or classes. Py Files Generator creates Python files and module subtrees with custom generators. The article summarizes by recommending the Python Extension Pack for overall productivity. It covers four categories: Python Language Pack, Jupyter, Django, and Project/Document generators. These extensions can boost productivity by 30%. Readers are invited to share their own favorite extensions.

NOTES



Further study materials Lesson 3

Transcript

Visual Studio Code 101 #5: 10+ Python Extensions Recommended

VSC Series

[Image: Visual Studio Code 101 #5: 10+ Python Extensions Recommended]

• Introduction • Story Series • Main Part on VSC Extension • 1. Python Language Extension Pack on VSC Extension • 2. Jupyter on VSC Extension • 3. Django on VSC Extension • 4. Project and Document Generator on VSC Extension • Summary

Introduction

Visual Studio Code is a powerful and free IDE that supports 10+ language development and a rich marketplace managed by Microsoft. In this story, we share our experience with 100+ extensions and cut the best Python extensions you can't miss. Moreover, we recommend these extensions to speed up your productivity by 30% and write your code with high efficiency.

In this section, we recommend you refer to the introduction on Python for Visual Studio Code, which shows how to use VS Code as a Python environment, primarily how to edit, run, and debug code through the VS Code IDE.

Finally, we are going to introduce useful extensions, and we hope these extensions can help speed up your working efficiency.

Main Part on VSC Extension

1. Python Language Extension Pack on VSC Extension

Get your Python perfect fit in VS Code IDE! First of all, I recommend you install the **Python Extension Pack** from "Don Jayamanne" if you don't have any idea about Python extensions.

Python Extension Pack

- CodeName: donjayamanne.python-extension-pack
- Description: Popular Visual Studio Code extensions for Python.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension for coding Python, and it contains the most popular Python extensions you need. All-in-one package. Truly recommended.
- Package: This extension includes the items below:
 - Python
 - Pylance
 - Jupyter

NOTES

Further study materials Lesson 3

Transcript

- Jinja
- Django
- Python Environment Manager
- Python Docstring Generator
- Python Indent

- Quick View:

[Image: Python Extension Pack extensions in Visual Studio Code]

Python

- CodeName: ms-python.python
- Description: IntelliSense (Pylance), Linting, Debugging (multi-threaded, remote), Jupyter Notebooks, code formatting, refactoring, unit tests, and more.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension for coding Python, and it adds language support for Python to Visual Studio Code, including features such as IntelliSense, linting, code formatting, debugging, testing, Jupyter Notebooks, Python environments, and refactoring. Truly recommended.
- Package: This extension includes the items below:
 - Python
 - Pylance
 - Jupyter
- Quick View:

[Image: Python extensions in Visual Studio Code]

Pylance

- CodeName: ms-python.vscode-pylance
- Description: A performant, feature-rich language server for Python in VS Code.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension as a language server for Python, and it is the default language support for Python in Visual Studio Code, shipped as part of that extension as an optional dependency. But note, it supports Python 3 better than Python 2.
- Quick View:

[Image: Pylance extensions in Visual Studio Code]

Python Snippets

- CodeName: frhtylcn.pythonsnippets

NOTES

Further study materials Lesson 3

Transcript

- Description: Code snippets for Python.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension for coding Python with code snippets support because I can't get this feature from the original Python extension. In the future, I think the Python extension from Microsoft will support Python snippets.
- Quick View:

[Image: python snippets extensions in Visual Studio Code]

Python Environment Manager

- CodeName: donjayamanne.python-environment-manager
- Description: View and manage Python environments & packages.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension to see how many Python virtual environments I have in my development environment. Most Python developers use conda, venv, pyenv, or other management tools. This extension gives you an all-in-one view of all Python environments. Truly recommended.
- Quick View:

[Image: Python Environment Manager extensions in Visual Studio Code]

Python Indent

- CodeName: KevinRose.vsc-python-indent
- Description: Correct Python indentation.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension to correct Python indentation, and I like how it resolves the annoying issues with Python indentation. Moreover, I can write Python more efficiently instead of being blocked by type errors. Truly recommended.
- Quick View:

[Image: Python Indent extensions in Visual Studio Code]

Python Type Hint

- CodeName: njqdev.vscode-python-typehint
- Description: Type hint completion for Python.
- Given Ranking: ★★★★★ 4 stars
- VSC Link: [Link](#)
- Comment: You can use this. I use this extension as an

NOTES

Further study materials Lesson 3

Transcript

alternative solution for type hint auto-completion in Python, with completion items for built-in types, classes, and the typing module. Just try it.

- Quick View:

[Image: Python Type Hint extensions in Visual Studio Code]

Python Test Explorer for Visual Studio Code

- CodeName: LittleFoxTeam.vscode-python-test-adapter
- Description: Run your Python tests in the Sidebar of Visual Studio Code.
- Given Ranking: ★★★★★ 4 stars
- VSC Link: [Link](#)
- Comment: You can use this. I use this extension as an alternative unit test solution, and I can easily run pytest or configure auto-run unit tests when I click save on a Python file. However, for me, I like running unit tests on the terminal like `pytest -s -lf -k test.py`. Anyway, just try it.
- Quick View:

[Image: Python Test Explorer extensions in Visual Studio Code]

2. Jupyter on VSC Extension

Jupyter

- CodeName: ms-toolsai.jupyter
- Description: Jupyter notebook support, interactive programming and computing that supports IntelliSense, debugging, and more.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension as an alternative Python prototype solution during the initial project stage, and I also love Jupyter Lab. For me, they are perfect IDEs with different design concepts. For you, just try and pick the one you like best.
- Quick View:

[Image: Jupyter extensions in Visual Studio Code]

Jupyter Keymap

- CodeName: ms-toolsai.jupyter-keymap
- Description: Jupyter keymaps for notebooks.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension to fit the keymap as a notebook because I am very used to it. It provides keymaps for notebooks in Visual Studio Code to match the

NOTES

Further study materials Lesson 3

Transcript

keybindings in Jupyter Notebook. This extension comes with the Jupyter extension for Visual Studio Code and can be disabled or uninstalled.

- Quick View:

[Image: Jupyter Keymap extensions in Visual Studio Code]

Jupyter Notebook Renderers

- CodeName: ms-toolsai.jupyter-renderers
- Description: Renderers for Jupyter Notebooks (with plotly, vega, gif, png, svg, jpeg, and other such outputs).
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension to show pictures or diagrams (gif, png, svg, jpeg, and more) when using data display or visualization in the IDE, so I can see these renderers with this extension.
- Quick View:

[Image: Jupyter Notebook Renderers extensions in Visual Studio Code]

3. Django on VSC Extension

Django

- CodeName: batisteo.vscode-django
- Description: Beautiful syntax and scoped snippets for perfectionists with deadlines.
- Given Ranking: ★★★★★ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension to code my Django project, and I like the features for snippets and improved syntax that help me edit the frontend template. Django is my first web project framework, and I love it.
- Quick View:

[Image: Django extensions in Visual Studio Code]

Djaneiro - Django Snippets

- CodeName: thebarkman.vscode-djaneiro
- Description: A collection of snippets for Django templates, models, views, fields & forms. Ported from Djaneiro for SublimeText.
- Given Ranking: ★★★★★ 4 stars
- VSC Link: [Link](#)
- Comment: You can use this. I use this extension as an alternative solution to provide code snippets in my Django project. Its collection supports richer code snippets than the Django extension. Another good solution for code snippets.

NOTES

Further study materials Lesson 3

Transcript

- Quick View:

[Image: Djaneiro - Django Snippets extensions in Visual Studio Code]

Jinja

- CodeName: wholroyd.jinja
- Description: Jinja template language support for Visual Studio Code.
- Given Ranking: ☆☆☆☆ 5 stars
- VSC Link: [Link](#)
- Comment: You should use this. I use this extension to code my Django project with Jinja templates. It adds language colorization support for the Jinja template language to VS Code. Nice extension.

- Quick View:

[Image: Jinja extensions in Visual Studio Code]

4. Project and Document Generator on VSC Extension

autoDocstring - Python Docstring Generator

- CodeName: njpwerner.autodocstring
 - Description: Beautiful syntax and scoped snippets for perfectionists with deadlines.
 - Given Ranking: ☆☆☆☆ 5 stars
 - VSC Link: [Link](#)
 - Comment: You should use this. I use this extension to generate Python documentation easily, so all I have to do is write the comment on my function or class. It quickly generates docstrings for Python functions, and it can fit with variables or sections in your doc. Have a try.
- Quick View:

[Image: autoDocstring - Python Docstring Generator extensions in Visual Studio Code]

Py Files Generator

- CodeName: ffaraone.pyfilesgen
 - Description: Generate Python files and module subtree.
 - Given Ranking: ☆☆☆ 4 stars
 - VSC Link: [Link](#)
 - Comment: You can use this. I use this extension to generate the project tree and Python modules, and I like the custom generators feature that lets me create any files in my projects. Have a try.
- Quick View:

[Image: Py Files Generator extensions in Visual Studio Code]

NOTES

Further study materials Lesson 3

Transcript

Summary

In this post, we recommend installing the Python Extension Pack for Python development and covering four parts of Python extension categories, cutting the best extensions you should try. Again, we recommend these extensions to speed up your productivity by 30% and write your code with high efficiency.

4 Parts Python Extension Category

1. Python Language Extension Pack on VSC Extension
2. Jupyter on VSC Extension
3. Django on VSC Extension
4. Project and Document Generator on VSC Extension

Python Extension Pack - This extension includes the following items:

1. Python
2. Pylance
3. Jupyter
4. Jinja
5. Django
6. Python Environment Manager
7. Python Docstring Generator
8. Python Indent

If you have any good ideas or recommend VSC extensions, please tell me! Thank you.

Most of the pictures are from each extension. They have the copyright.

NOTES

Further study materials Lesson 3

Content type: Text

Setting Up a Python Development Environment in VS Code with Virtual Environments and Tools

Available at
<https://oandersonbm.medium.com/usando-python-no-vs-code-com-ambiente-virtual-venv-ecef7959b652>

Full
Content



Summary

Switching from PyCharm to VS Code

After using editors like Atom and PyCharm, the author recently started using Visual Studio Code professionally for PHP and Flutter/Dart development. They were pleased with VS Code's facilitators and lightness. Because it is highly customizable with many extensions, they decided to switch from PyCharm to VS Code for Python development. This article explains how to set up Python with a virtual environment in VS Code.

Prerequisites and Initial Setup

Prerequisites include knowledge of Python, Python 3, and installing VS Code with Python and Code Runner extensions. Create a test directory and open it in VS Code. In the terminal, run `'python3 -m venv venv'` to create a virtual environment. If using Anaconda, specify full path to `python.exe`. After creation, find the activate script in Scripts or bin folder.

Activating the Virtual Environment

To activate the virtual environment, use the command `'venv/Scripts/activate'` on Windows or `'venv/bin/activate'` on Unix. To deactivate, type `'deactivate'`. Then create a `main.py` file in the root directory. In VS Code, click on the Python version at the bottom left corner and select the interpreter that includes the word `'venv'`. This will create a `.vscode` folder with a `settings.json` file. Now the terminal will open with the `venv` configured automatically.

Configuring Code Runner for venv

Even after selecting the `venv` interpreter, Code Runner may still use the default Python. To fix, open `.vscode/settings.json` and add `'code-runner.executorMap'` with the `venv's python.exe` path, e.g. `'venv\Scripts\python.exe'`. Also set `'python.pythonPath'` to the same. This ensures code runs with the correct Python version.

Optional Settings and Conclusion

Additional configurations can be added to global settings or `.vscode/settings.json`. For example, enable `'code-runner.runInTerminal'` to show output in Terminal tab. For testing, enable `mypy` and `flake8` linting and `unittest`. The project

NOTES

Further study materials Lesson 3



Summary

structure included src and tests folders. Tests run directly from VS Code. The author recommends reading official documentation for more productivity tips.

NOTES

Further study materials Lesson 3

Transcript

Using Python in VS Code with a Virtual Environment (venv)

After working with various editors like Atom and PyCharm, I recently started using **Visual Studio Code** professionally for PHP and Flutter/Dart development. And I was very pleased with the facilitators and lightness of VS Code.

It is highly customizable and has many extensions. That's why I decided to switch from PyCharm to it.

[Image: https://miro.medium.com/0*Owmtc1n_1B5tjmpl.png]

PREREQUISITES

- Knowledge of Python
- Python 3
- Download and install VS Code
- Install the Python extension for VS Code
- Install the Code Runner extension for VS Code

Create a directory/folder on your computer to run tests, with any name. Then open the directory with VS Code.

[Image: https://miro.medium.com/1*fQqn4KFwc_USVArUcyKkzg.png]

Open the terminal:

[Image: https://miro.medium.com/1*dhsbtKnDCTkxXBtLZFMoxg.png]

Type:

```
python3 -m venv venv
```

Or, in case you prefer to use Anaconda, Miniconda, etc.:

```
C:\[PYTHON DIRECTORY]\python.exe -m venv venv
```

Then the environment will be almost configured regarding the *venv*. It is important to check how the configuration of your *venv* folder looks, because it might be different:

[Image: https://miro.medium.com/1*QDlelZsfY58YBim1jnC0jQ.png]

Mine has the activate in *Scripts*, but I have seen some people say it is in the *bin* folder. So type:

```
. venv/Scripts/activate  
# OR  
. venv/bin/activate
```

And to deactivate, type:

NOTES

Further study materials Lesson 3

Transcript

deactivate

Create a `main.py` file in your root directory. You will see in the bottom left corner the Python version you are using. Click on it:

[Image: https://miro.medium.com/1*Vrs8dA6GF3KcA6RdXf_MuA.png]

Regardless of how many versions and folders appear (in my case there are several), select the one that has the word `venv`:

[Image: https://miro.medium.com/1*onOgUIMQfg2c4aBMdAwcbQ.png]

Then VS Code will create a configuration directory called `.vscode`:

[Image: https://miro.medium.com/1*xZTdfHd1Tzvd2KjpR_cPtw.png]

Now, whenever you open the terminal for this project, it will open with the `venv` configured. However, there is still a small configuration needed so that it calls Python from the `venv` directory/folder. In my case, I have version 2.7 installed as default, but the `venv` one is 3.7. When I type:

```
import sys
print(sys.version)
```

It returns `2.7.18 (v2.7.18:8d21aa21f2, Apr 20 2020, 13:25:05) [MSC v.1500 64 bit (AMD64)]`, in other words, "nothing at all" because I want it to use the `venv`.

[Image: https://miro.medium.com/0*c8hTgk9KV5vHdJF2.gif]

Well, let's fix this thing. Open the `.vscode` directory and open the `settings.json` file, then modify it by adding `code-runner.executorMap`. The file will look like this:

```
{
  "python.pythonPath": "venv\\Scripts\\python.exe",
  "code-runner.executorMap": {
    "python": "venv\\Scripts\\python.exe"
  }
}
```

Now your code will run normally.

Additional/Optional Configurations

From now on, you can add all configurations either in the global settings (image below) or in your `settings.json` file inside the `.vscode` folder:

[Image: https://miro.medium.com/1*xjAiwNj_mqfmG7DI1zoN0Q.png]

[Image: https://miro.medium.com/1*7r7yVR4skLI0vDkMNE50Q.png]

NOTES

Further study materials Lesson 3

Transcript

I recommend adding only what you want to be already configured for future projects to the global file.

Displaying Python in the Terminal Tab

You can also add `code-runner.runInTerminal` to make it display in the `Terminal` tab instead of the `Console`:

```
"code-runner.runInTerminal": true
```

Configuring Tests

I deleted the `main.py` file and made the following project division so that we can configure tests automatically in VS Code. It ended up like this (**pycache** generated automatically):

[Image: https://miro.medium.com/1*7Srx2nDqo9KqXlfr1ubgbg.png]

In the settings, I added:

```
"python.linting.mypyEnabled": true,  
"python.linting.flake8Enabled": true,  
"python.testing.unittestEnabled": true,  
"[python]": {  
  "editor.formatOnSave": true  
}
```

The files `tests/__init__.py` and `src/main.py` ended up like this:

And you can run the tests directly from VS Code:

[Image: https://miro.medium.com/1*Azsaq8FVT35X1gvVP3mn_A.png]

Conclusion

In this article, I showed how to use VS Code with Python using some plugins and configurations. I may update this or complement it in another tutorial. Either way, I recommend reading the official documentation because some other plugins might make you more productive.

Thank you for your visit, and I hope I have helped you. Hugs and see you soon!

[Image: [https://miro.medium.com/0*4ou05RncqEfZVAV\].gif](https://miro.medium.com/0*4ou05RncqEfZVAV].gif)]

NOTES

Exercises • Lesson 3

EXERCISE 1 • True or false

CONTEXT

Configuring VS Code properly for Python and Flet development involves setting up extensions, selecting the correct interpreter, enabling formatting, and running scripts from the editor.

QUESTION • Indicate whether the information is true or false

For Flet development in VS Code, the only required extension is the Python extension.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 2 • Multiple choice

CONTEXT

To maintain code consistency in a Flet project, developers often use automatic formatting tools.

QUESTION • Select the correct option

What is the function of Black, the formatter configured in Step 3 of the VS Code setup?

- ☐ A It automatically reformats code on save to enforce a consistent style.
- ☐ B It installs Python packages needed for Flet.
- ☐ C It checks for syntax errors in the code.
- ☐ D It runs the Flet app in the terminal.

NOTES

Exercises • Lesson 3

EXERCISE 3 • True or false

CONTEXT

In many software teams, debates about code formatting—such as quotes, indentation, and line breaks—can consume significant time and energy. Tools like Black aim to address this issue.

QUESTION • Indicate whether the information is true or false

Black is an opinionated formatter that does not provide configuration options for personal style preferences.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 4 • Multiple choice

CONTEXT

In the lesson, Black is described as an opinionated code formatter that aims to eliminate formatting debates and standardize Python code.

QUESTION • Select the correct option

What is the primary purpose of Black?

- ☐ A To manage project dependencies and virtual environments.
- ☐ B To detect and fix bugs in Python code.
- ☐ C To optimize code performance by rewriting inefficient patterns.
- ☐ D To automatically format Python code according to a fixed set of rules, ensuring uniform style.

NOTES

Exercises • Lesson 3

EXERCISE 5 • True or false

CONTEXT

In Python development, various tools help maintain code quality: isort sorts imports, Black formats code, and Flake8 checks for style issues.

QUESTION • Indicate whether the information is true or false

Flake8 is used to automatically format Python code.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 6 • Multiple choice

CONTEXT

When managing Python project settings, developers can use various configuration file formats. One format has become the new standard for tools like Black, isort, and Flake8.

QUESTION • Select the correct option

Which configuration file format is considered the new standard for Python project configuration?

- ☐ A YAML
- ☐ B TOML
- ☐ C INI
- ☐ D JSON

NOTES

Exercises • Lesson 3

EXERCISE 7 • True or false

CONTEXT

The June 2025 update to the Python extension in VS Code introduced several new features, including improvements to the interactive REPL experience.

QUESTION • Indicate whether the information is true or false

The REPL completions powered by the Python Language Server are disabled by default in this update.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 8 • Multiple choice

CONTEXT

In the June 2025 update, Microsoft introduced several features to improve the Python development experience in VS Code. One of these features helps users quickly bootstrap new projects with preconfigured structures and dependencies.

QUESTION • Select the correct option

What does the 'Create Project from Template' command allow users to do?

- ☐ A It relies on cookiecutter templates downloaded from the internet at the time of creation.
- ☐ B It offers a choice of predefined project templates (e.g., Flask, FastAPI, data science) and sets up folders, requirements files, and optionally pyproject.toml, with the option to select an environment tool.
- ☐ C It only creates a basic script project with a single main.py file and no additional configuration.
- ☐ D It requires users to manually install all packages after the project is created, as no requirements files are generated.

NOTES

Exercises • Lesson 3

EXERCISE 9 • True or false

CONTEXT

Python code formatters like Black and Ruff help maintain consistent code style. They automate formatting according to PEP 8 guidelines.

QUESTION • Indicate whether the information is true or false

Black code formatter can be configured to use single quotes instead of double quotes.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 10 • Multiple choice

CONTEXT

In the lesson, two Python code formatters are compared: Black and Ruff. One of the key differences is their performance and configurability.

QUESTION • Select the correct option

What is a notable advantage of Ruff over Black as mentioned in the lesson?

- ☐ A Ruff is faster because it is written in Rust.
- ☐ B Ruff cannot be used with pre-commit hooks.
- ☐ C Ruff uses double quotes by default, while Black uses single quotes.
- ☐ D Ruff is only compatible with Python 2.7 and earlier.

NOTES

Exercises • Lesson 3

EXERCISE 11 • True or false

CONTEXT

The Python Extension Pack is a collection of popular Visual Studio Code extensions for Python development.

QUESTION • Indicate whether the information is true or false

The Python Extension Pack includes the Python Type Hint extension.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 12 • Multiple choice

CONTEXT

The Python Extension Pack is a popular collection of extensions for Python development in VS Code, as recommended in the lesson.

QUESTION • Select the correct option

How many extensions are included in the Python Extension Pack?

- ☐ A 9
- ☐ B 6
- ☐ C 8
- ☐ D 7

NOTES

Exercises • Lesson 3

EXERCISE 13 • True or false

CONTEXT

Setting up a virtual environment in VS Code involves creating the environment and selecting the interpreter to use.

QUESTION • Indicate whether the information is true or false

Once a virtual environment is created and activated in VS Code, Python code will automatically run using the virtual environment's interpreter without any additional configuration.

☐ TRUE

☐ FALSE

NOTES

Exercises • Lesson 3

EXERCISE 14 • Multiple choice

CONTEXT

After creating a virtual environment in a VS Code project and activating it, you may notice that running Python code still uses the system's default Python interpreter. To fix this for the Code Runner extension, you need to configure a specific setting.

QUESTION • Select the correct option

What setting should be added to the `.vscode/settings.json` file to ensure the Code Runner extension uses the Python interpreter from the virtual environment?

- ☐ A Set 'python.pythonPath' to the venv's python.exe.
- ☐ B Set 'python.linting.flake8Enabled' to true.
- ☐ C Add 'code-runner.executorMap' with the path to the venv's python.exe.
- ☐ D Set 'code-runner.runInTerminal' to true.

NOTES

Exercise Answers

LESSON 3

EX.01 • True or false

False.

The lesson states that three extensions are essential: Python, Pylance, and Black Formatter. The Python extension alone is insufficient for full development support.

☒ I got it right ☐ I got it wrong

EX.02 • Multiple choice

Option A — “It automatically reformats code on save to enforce a consistent style.”

The lesson states that Black is an opinionated formatter that reformats your code every time you save the file, ensuring consistency without any configuration.

☒ I got it right ☐ I got it wrong

EX.03 • True or false

True.

Black is intentionally opinionated, imposing a single non-negotiable style. It offers very few configuration options to eliminate debates and ensure consistency across all codebases.

☒ I got it right ☐ I got it wrong

EX.04 • Multiple choice

Option D — “To automatically format Python code according to a fixed set of rules, ensuring uniform style.”

Black removes personal style choices by deterministically formatting code, making all code look the same and ending discussions.

☒ I got it right ☐ I got it wrong

EX.05 • True or false

False.

Flake8 is a linter that checks for style and syntax errors, but it does not automatically format code. Black is the tool that reformats code automatically.

☒ I got it right ☐ I got it wrong

EX.06 • Multiple choice

Option B — “TOML”

TOML is highlighted as the new standard for Python project configuration, supported by tools like Black, isort, Flake8, Poetry, and Hatch. It is easy to read and supports nested tables.

☒ I got it right ☐ I got it wrong

Exercise Answers

LESSON 3

EX.07 • True or false

False.

The lesson states that the REPL completions are enabled by default, and users can disable them by adding a setting. Therefore, the statement that they are disabled by default is false.

☒ I got it right ☐ I got it wrong

EX.08 • Multiple choice

Option B — “It offers a choice of predefined project templates (e.g., Flask, FastAPI, data science) and sets up folders, requirements files, and optionally pyproject.toml, with the option to select an environment tool.”

The lesson states that the command prompts users to choose from templates like Flask, FastAPI, or data science, and initializes with folders, .gitignore, requirements.txt, and optionally pyproject.toml, along with environment tool selection (venv, conda, Poetry).

☒ I got it right ☐ I got it wrong

EX.09 • True or false

False.

Black uses double quotes and does not support a configuration to switch to single quotes. Only Ruff has a configuration option for quote style via its ruff.toml file.

☒ I got it right ☐ I got it wrong

EX.10 • Multiple choice

Option A — “Ruff is faster because it is written in Rust.”

The lesson explicitly states that Ruff is written in Rust and is 30x faster than Black, making speed a key advantage.

☒ I got it right ☐ I got it wrong

EX.11 • True or false

False.

The Python Extension Pack includes Python, Pylance, Jupyter, Jinja, Django, Python Environment Manager, Python Docstring Generator, and Python Indent, but not Python Type Hint.

☒ I got it right ☐ I got it wrong

EX.12 • Multiple choice

Option C — “8”

The lesson clearly lists eight extensions included in the Python Extension Pack: Python, Pylance, Jupyter, Jinja, Django, Python Environment Manager, Python Docstring Generator, and Python Indent.

☒ I got it right ☐ I got it wrong

Exercise Answers

LESSON 3

EX.13 • True or false

False.

Even after activating the virtual environment, VS Code's Code Runner may still use the system Python interpreter. To force it to use the virtual environment's interpreter, you must modify the '.vscode/settings.json' file to set the 'python.pythonPath' and 'code-runner.executorMap' entries.

☒ I got it right ☐ I got it wrong

EX.14 • Multiple choice

Option C — “Add 'code-runner.executorMap' with the path to the venv's python.exe.”

The lesson explains that adding 'code-runner.executorMap' with 'python': 'venv\\Scripts\\python.exe' directs Code Runner to use the virtual environment's interpreter.

☒ I got it right ☐ I got it wrong

Chapter 01 • Introduction to Flet and Python Setup

Practical project of the chapter

14 pages

Practical project of the chapter

14 pages

Context and Provocation	137
Development Guide	140
Self-Assessment Rubric	140
Model Response & Assessment Reference	141
Notes	147

Introduction to Flet and Python Setup

Driving Question

A screenshot or short video of your first Flet app displaying a welcome message.

Production Format

Argumentative essay — 800 to 1500 words.

Context and Provocation

Objective

You will set up your Python and Flet development environment, write a minimal Flet application, and capture proof that your toolchain is ready for the course. This exercises core competencies: environment setup, Flet app structure, Page object, adding a Text control, and running with hot reload.

Creative Brief

Your friend has commissioned you to build a personal journal app (the course project). But before they trust you with their deepest thoughts, they want to see that you can actually run a Flet app. "Just send me a quick video of a window that says something like 'Flet is ready!' or your name – I want to know the tools work." This is a small but crucial milestone: proving your development environment is functional and you understand the basics. Deliver a polished, minimal greeting app.

Objective

You will set up your Python and Flet development environment, write a minimal Flet application, and capture proof that your toolchain is ready for the course. This exercises core competencies: environment setup, Flet app structure, Page object, adding a Text control, and running with hot reload.

Creative Brief

Your friend has commissioned you to build a personal journal app (the course project). But before they trust you with their deepest thoughts, they want to see that you can actually run a Flet app. "Just send me a quick video of a window that says something like 'Flet is ready!' or your name – I want to know the tools work." This is a small but crucial milestone: proving your development environment is functional and you understand the basics. Deliver a polished, minimal greeting app.

Materials and Resources

- Computer with Python 3.8+ installed
- Internet connection to install Python and Flet
- VS Code or PyCharm (optional but recommended)
- Screen recording software (e.g., OBS Studio, Game Bar, QuickTime) or screenshot tool

Execution Steps

1. Install Python 3.8 or higher if not already installed.
 - Professional insider tip: Check your Python version with `'python --version'` or `'python3 --version'`. If you have multiple versions, create a virtual environment to avoid conflicts.
 - Common mistake to avoid: Don't skip the virtual environment; it prevents package conflicts later.
2. Create a new project folder and set up a virtual environment.
 - Professional insider tip: Use `'python -m venv .venv'` and activate it. In VS Code, the interpreter will auto-select if you open the folder.
 - Common mistake to avoid: Forgetting to activate the environment before installing packages.
3. Install Flet using pip: `'pip install flet'`.
 - Professional insider tip: Use `'pip install --upgrade pip'` first to ensure you get the latest package.
 - Common mistake to avoid: Running pip without the virtual environment activated may install globally.
4. Write a minimal Flet application in a file named `'first_app.py'`.
 - Professional insider tip: Start with `'import flet as ft'` and define a `'main'` function that receives `'page'`. Add a `'Text'` control with your greeting.
 - Common mistake to avoid: Don't forget to call `'ft.app(target=main)'` at the bottom.
5. Run the app using `'python first_app.py'`. A window should appear with your message.
 - Professional insider tip: For this simple app, just run normally. If you change code while running, save and Flet will reload automatically.
 - Common mistake to avoid: Closing the terminal stops the app; keep it open.
6. Capture a screenshot or a short screen recording of the running app showing your greeting.
 - Professional insider tip: If recording, keep it under 15 seconds. Show the code and then the running window.
 - Common mistake to avoid: Don't record just the code; ensure the running window is visible.

Self-Assessment Checklist

- My Python version is 3.8 or higher.
- I created and activated a virtual environment.
- Flet is installed without errors.
- My app runs and displays a Text control with a greeting.
- The app window appears and closes properly (no crash).
- I have a screenshot or video file that clearly shows the app's window with the greeting.
- (Optional) I used hot reload to see a change instantly.

Bonus Challenge

Customize the greeting with your name, a background color, or a fun emoji using 'Text' properties like 'size', 'color', 'bgcolor'. Or add a second text line.

Submission Format

Accepted formats:

- image (.png/.jpg/.webp/.heic)
- video (.mp4/.mov/.webm)

Minimum delivery:

- A single image file showing your running Flet app.

Development Guide

1. Research Phase

Look for examples of small apartment designs on sites like ArchDaily or Dezeen. Read news articles on micro-apartments to see both praise and criticism. Note how different designs handle the space-saving vs. comfort tradeoff. Use the course chapters on apartment planning and 3D modeling as your main guides.

2. Organization Phase

Start by stating your main argument — your position on balancing efficiency and comfort. Then outline three to four key points that support your view, using the mandatory references. For each point, plan what evidence or examples you'll use. Also think about one counterargument and how you'll respond to it.

3. Writing Phase

Write an introduction that presents the problem and your thesis. In the body paragraphs, each should focus on one key point — explaining a course concept and applying it to your design example. Use simple language: imagine explaining to a friend. Include specific examples, like how you'd place a bed or choose lighting. End with a conclusion that summarizes your argument and suggests practical tips for designers.

4. Review Phase

Check that your essay clearly states your position. Verify that you've used all four mandatory references correctly. Read it aloud to see if the logic flows smoothly. Ask yourself: would a beginner understand this? Remove any jargon or complex terms without explanation.

Self-Assessment Rubric

1. App runs and displays greeting

- Insufficient — App crashes, no window, or missing text
- Adequate — Text visible, plain, no errors
- Excellent — Text visible, styled (color/size/emoji), no errors

2. Correct code structure

- Insufficient — Missing one of the core parts (e.g., no `ft.app` call)
- Adequate — Same but minor formatting issues
- Excellent — Imports `flet`, defines `main`, adds `Text` control, calls `ft.app(target=main)` properly

3. Screenshot/video quality

- Insufficient — Not showing window, too dark, or cropped
- Adequate — Clear enough to see greeting but small/blurry
- Excellent — High resolution, entire window visible, greeting readable

4. Environment setup (Python version, virtual environment, Flet install)

- Insufficient — App fails due to missing dependencies or wrong Python version
- Adequate — Not shown but app runs (inferred)
- Excellent — Evidence of virtual environment and Python version (screenshot of terminal)

Model Response & Assessment Reference

This guide presents possible paths, not single answers. An excellent essay may defend any of the theses below — or an original thesis not anticipated here — as long as it demonstrates argumentative rigor, correct use of sources, and autonomous critical reflection.

1. Expected Thesis Position(s)

The central question admits multiple legitimate positions. Below are argumentative paths representing robust and distinct directions.

Author note: *The recommended range is 2 to 4 theses. If the assignment has only one defensible answer, keep just Thesis A and adapt the wording. A third thesis often works best as a synthetic or dialectical position.*

1.1. Thesis A — {Short label}

{Full thesis statement, in one or two sentences. It should take a clear position on the central question.}

Main argumentative line: {One paragraph describing how this thesis sustains itself: the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.2. Thesis B — {Short label}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {One paragraph describing the central claim, supporting reasoning, implicit framework, and what makes it defensible.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

1.3. Thesis C — {Synthetic/dialectical position, optional}

{Full thesis statement — a meaningfully different position from Thesis A, not just a softer variant.}

Main argumentative line: {Show explicitly how this position produces a new insight that neither A nor B alone can reach.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2. Key Concepts and Their Correct Use

Author note: Include 3 to 5 key concepts per project. For each concept: precise definition + example of correct application + common pitfall. The pitfall section is critical — it tells the evaluator what to watch for.

2.1. A — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work — not just being named, but actively grounding a claim.}

Common pitfall: {The most frequent way students misuse this concept: over-application, decorative citation, missing nuance, ignoring critiques, or conflating related concepts. Be concrete.}

2.2. B — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

2.3. C — {Concept Name}

How to define/explain: {One paragraph defining the concept precisely. Include its theoretical origin or author when relevant. Define its scope and limits.}

Correct use (example): {A verbatim example paragraph showing the concept doing actual argumentative work and grounding a claim.}

Concepts mobilized: {Course concept 1} ({brief connection}); {Course concept 2} ({brief connection}); {Course concept 3} ({brief connection}).

3. Model Argumentative Structure

The outline below illustrates the structure of an excellent-level essay, regardless of which thesis the student chooses.

Author note: Word ranges are suggestions calibrated for a {total length}-word essay. Adjust proportionally if the assignment is shorter or longer.

3.1. Introduction ({X}–{Y} words)

Contemporary hook: {Describe what kind of opening works for this project — a concrete example, striking data point, scene from culture, or current event. Adapt to the project's domain.}

Bridge to subject: {How the hook should pivot into the project's central question.}

Explicit thesis: The author's position in one or two clear, specific sentences, positioned directly against the central question.

3.2. Development ({X}–{Y} words)

Suggested: 2–3 paragraphs, each with a central argument.

Paragraph 1: {Function and how it advances the thesis.}

Paragraph 2: {Function and how it advances the thesis.}

Paragraph 3: {Function and how it advances the thesis.}

3.3. Counterargument ({X}–{Y} words)

An excellent essay does not merely mention an objection: it presents the strongest counterargument with intellectual honesty and responds consistently.

Author note: *List the best counterargument for each thesis option, with a suggested response. The student should not feel that confronting the objection weakens their position — done well, it strengthens it.*

If thesis is A: {Strongest objection and possible response.}

If thesis is B: {Strongest objection and possible response.}

If thesis is C: {Strongest objection and possible response.}

3.4. Conclusion ({X}–{Y} words)

Restatement of thesis: Reaffirm the position in light of the full argumentation, without repeating the introduction verbatim.

Implications: {What this reflection reveals about something larger — the field, how we think, or a contemporary issue.}

Opening (no new information): A provocative question or forward-looking gesture that broadens the horizon without introducing arguments not developed in the essay.

3.5. Evidence and Data Types to Mobilize

- Specific passages from primary sources.
- Theoretical concepts with citation to the original work.
- Documented contemporary examples: news, official reports, data, or cultural productions.
- References to academic debates covered in the course.

4. Rubric — Excellent Level

Detailed description of what the evaluator should observe in an essay rated “Excellent” on each criterion.

Author note: *The criteria below cover the standard dimensions of argumentative essays. Remove or adapt criteria that do not apply to the specific project.*

4.1. Thesis clarity

What is observed at the EXCELLENT level: The thesis appears in the introduction unequivocally. It is specific and directly engages the project's central tension. The entire essay is organized around it.

4.2. Use of course concepts

What is observed at the EXCELLENT level: Required references are defined precisely before application. Concepts are integrated organically into the argument, and their limits are recognized.

4.3. Quality of argumentation

What is observed at the EXCELLENT level: Each paragraph opens with a clear claim, sustains it with evidence, and connects back to the thesis. Logical progression is clear and the student analyzes rather than merely describes.

4.4. Counterarguments

What is observed at the EXCELLENT level: The student presents the strongest objection to their position and responds with a substantial argument. The thesis emerges stronger from confronting the critique.

4.5. Originality of analysis

What is observed at the EXCELLENT level: The essay goes beyond reproducing course materials. The student offers original connections, asks new questions, or proposes a novel interpretation from familiar evidence.

4.6. Connection to the contemporary

What is observed at the EXCELLENT level: The dialogue between subject and present is specific and grounded. Concrete contemporary examples illuminate both sides.

4.7. Structure and academic norms

What is observed at the EXCELLENT level: Cohesive structure with fluid transitions. Citations follow the established academic pattern, and references are complete and correctly formatted.

5. Common Argumentative Errors

The errors below are the most frequent in this type of project. The evaluator should watch for them.

Author note: Errors 2, 3, and 4 apply to almost any argumentative project. Errors 1 and 5 are project-specific slots where particular risks may be described.

5.1. {Project-specific error}

How it appears in the text: {Verbatim-style example showing the error in action.}

How to correct: {Concrete, actionable advice on what to do differently.}

5.2. Strawman fallacy

How it appears in the text: The student presents the counterargument in its weakest, easiest-to-refute form instead of confronting the strongest version.

How to correct: Reformulate the counterargument as an intelligent interlocutor would present it, then respond to that reason.

5.3. Decorative use of concepts

How it appears in the text: The student names a concept but never applies it — it appears as a loose citation, disconnected from the argument.

How to correct: After citing a concept, explain exactly how it grounds the specific argument of that paragraph.

5.4. False equivalence

How it appears in the text: The student treats as equivalent two phenomena that operate in radically different contexts or functions.

How to correct: Make differences explicit before drawing parallels. Use precise comparative language. Comparison is not equation.

5.5. {Project-specific error}

How it appears in the text: {Example}

How to correct: {Correction}

Final reminder: *The quality of reasoning is always more important than the position defended. An essay that defends an unpopular thesis with rigor outperforms one that defends a consensus thesis superficially.*

